

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1563

**PRIMJENA AIOPS METODOLOGIJE ZA
DETEKCIJU ANOMALIJA U GITHUB ACTIONS
CJEVOVODIMA**

Dominik Ožvald

Zagreb, ožujak 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 1563

**PRIMJENA AIOPS METODOLOGIJE ZA
DETEKCIJU ANOMALIJA U GITHUB ACTIONS
CJEVOVODIMA**

Dominik Ožvald

Zagreb, ožujak 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI ZADATAK br. 1563

Pristupnik: **Dominik Ožvald (0036543658)**

Studij: Računarstvo

Profil: Računarska znanost

Mentor: izv. prof. dr. sc. Vlado Sruk

Zadatak: **Primjena AIOps metodologije za detekciju anomalija u GitHub Actions cjevovodima**

Opis zadatka:

Moderni razvojno-operativni procesi (DevOps) oslanjaju se na automatizirane procese kontinuirane integracije i isporuke (CI/CD), koji u praksi postaju sve složeniji zbog rasta broja ovisnosti, razgranatosti ispitnih slučajeva i sve češće uporabe ponovo iskoristivih predložaka. Posljedično, kvarovi izvođenja, degradacije vremena izvođenja i regresije pouzdanosti često se otkrivaju kasno te zahtijevaju ručnu analizu velikih količina logova i metapodataka. Dodatno, određeni obrasci anomalija mogu upućivati na povećan operativni rizik, zbog čega je važna rana detekcija. Cilj ovog rada je osmisliti, implementirati i eksperimentalno evaluirati pristup temeljen na umjetnoj inteligenciji za IT operacije (AIOps) za automatizacijski sustav GitHuba (GitHub Actions) koje automatizira analizu telemetrije CI/CD cjevovoda i detektira anomalije te degradacije performansi. Potrebno je odabrati i analizirati dostupne podatke iz GitHub Actions okruženja te definirati i formalizirati relevantne tipove anomalija. Na temelju toga razraditi postupak izgradnje skupa podataka i značajki te implementirati modele za detekciju anomalija. Razmotriti zahtjeve integracije u razvojni proces kroz automatizirani izvještaj ili obavijesti. Provesti empirijsku evaluaciju na reprezentativnom skupu podataka, uz analizu točnosti detekcije i robusnosti.

Rok za predaju rada: 26. lipnja 2026.

Sažetak

Naslov: Primjena AIOps metodologije za detekciju anomalija u GitHub Actions cjevovodima

Sažetak: U radu je osmišljen, implementiran i evaluiran GHA-AIOps pristup za automatiziranu detekciju anomalija u zapisima GitHub Actions radnih tokova. Razmatrane su četiri vrste anomalija: nestabilni testovi, konfiguracijski pomak, tihi neuspjesi i sigurnosne anomalije. Za učenje i evaluaciju korišteni su umjetno generirani skup s kontrolirano umetnutim anomalijama i označeni podskup stvarnih zapisa GHALogs. Zapisi su vektorizirani autoenkoderom i obrađeni dvama modelima transformera: višeklasnim modelom za određivanje vrste anomalije i rekonstrukcijskim modelom za binarnu detekciju. Pristup je integriran u GitHub Actions uz automatsko izvještavanje. Višeklasni model ostvario je makro F1-mjeru od 0,87227 na umjetno generiranom skupu i 0,68736 na podskupu GHALogs, dok je rekonstrukcijski model ostvario AUC od 0,8702. Rezultati pokazuju visoku uspješnost na kontroliranom skupu, dok rezultati na GHALogs podskupu upućuju na ograničenu prenosivost na stvarne, heterogenije zapise.

Ključne riječi: AIOps, autoenkoder, detekcija anomalija, GitHub Akcije, transformer

Summary

Title: Application of AIOps methodology for anomaly detection in GitHub Actions

Summary: In this paper the GHA-AIOps approach for automated anomaly detection in GitHub Actions logs of workflows is designed, implemented and evaluated. Four types of anomalies are considered: flaky tests, configuration drift, silent failures, and security anomalies. An artificially generated set with controlled inserted anomalies and a labeled subset of real GHALogs logs were used for learning and evaluation. Records were vectorized with an autoencoder and processed with two transformer models: a multi-class model for determining the type of anomaly and a reconstruction model for binary detection. The approach is integrated into GitHub Actions with automatic reporting. The multiclass model achieved a macro F1-measure of 0,87227 on the artificially generated set and 0,68736 on the GHALogs subset, while the reconstruction model achieved an AUC of 0,8702. Results show high performance on the controlled set, while results on the GHALogs subset suggest limited transferability to real, more heterogeneous logs.

Keywords: AIOps, autoencoder, anomaly detection, GitHub Actions, transformer

Sadržaj

Sažetak	5
Summary	6
Popis slika	1
Popis Tablica	2
Kazalo kratica i oznaka	3
1. Uvod	5
2. Pregled AIOps pristupa i anomalija u CI/CD zapisima	8
2.1. AIOps pristupi detekciji anomalija u razvojno-operativnim procesima	8
2.2. Tipovi anomalija u CI/CD zapisima	10
2.2.1. Kvarovi i degradacije u CI/CD cjevovodima	11
2.2.2. Specifičnosti GitHub Actions radnih tokova i odabrane klase anomalija	12
3. Podatkovna osnova GHA-AIOps pristupa	16
3.1. Ulazni podaci i izvori zapisa	16
3.1.1. Struktura zapisa GitHub Akcija i metapodataka	16
3.1.2. Umjetno generirani skup podataka	18
3.1.3. Skup podataka GHALogs	18
3.2. Preslikavanje anomalija na zapise i označavanje zapisa	19
3.2.1. Preslikavanje tipova anomalija na zapise radnog toka	19
3.2.2. Označavanje umjetno generiranih zapisa	21
3.2.3. Označavanje stvarnih GHALogs zapisa	21
3.3. Predobrada zapisa, izgradnja značajki i podjela podataka	21
3.3.1. Predobrada tekstualnih zapisa	21
3.3.2. Pretvaranje zapisa u vektore značajki	22
3.3.3. Oblikovanje kliznih prozora	22
3.3.4. Podjela podataka na skupove za učenje, prilagodbu i ispitivanje	23
3.4. Tenzorska reprezentacija podataka i neravnoteža klasa	24

4.	Implementacija GHA-AIOps pristupa za detekciju anomalija	26
4.1.	Zahtjevi i prikaz implementiranog rješenja	26
4.2.	Kodiranje zapisa autoenkoderom	28
4.2.1.	Arhitektura autoenkodera	28
4.2.2.	Učenje autoenkodera i vektorski prikaz zapisa	30
4.3.	Modeli transformera za detekciju anomalija	31
4.3.1.	Višeklasni model transformera	33
4.3.2.	Rekonstrukcijski model transformera	33
4.3.3.	Učenje modela transformera	34
4.3.4.	Uloga dvaju modela u GHA-AIOps pristupu	36
4.4.	Integracija u GitHub Actions i ograničenja izvedbe	36
4.4.1.	Organizacija programskih komponenti	37
4.4.2.	Dohvat zapisa i pokretanje detekcije	40
4.4.3.	Izrada izvještaja i slanje obavijesti	41
4.4.4.	Ograničenja izvedbe	44
5.	Eksperimentalna evaluacija GHA-AIOps pristupa	46
5.1.	Metodologija evaluacije	46
5.1.1.	Eksperimentalni protokol	47
5.1.2.	Mjere evaluacije	48
5.2.	Evaluacija višeklasnog modela	51
5.2.1.	Rezultati na umjetno generirani skup podataka	51
5.2.2.	Rezultati na skupu podataka GHALogs	53
5.2.3.	Analiza pogrešaka višeklasnog modela	57
5.3.	Evaluacija rekonstrukcijskog modela	58
5.4.	Rasprava rezultata i ograničenja	60
6.	Zaključak	62
	Literatura	64

Dodatak A: Repozitorij, izvorni kod i reprodukcija rezultata.....	67
A.1 struktura	67
A.2 Upute za pokretanje i programska okolina	69
Dodatak B: LSTM ćelija u koderu i dekoderu autoenkodera	70
Dodatak C: Arhitektura transformera u modelima za detekciju anomalija	73

Popis slika

Slika 2.1 Konceptualni prikaz mogućih mjesta pojave promatranih anomalija u GitHub Actions radnom toku.	14
Slika 3.1 Hijerarhija izvođenja i generiranja podataka unutar GitHub Actions radnog toka	17
Slika 3.2 Prikaz kliznog prozora.....	23
Slika 4.1 Arhitektura autoenkodera	29
Slika 4.2 Prikaz LogBERT Arhitekture [24]	32
Slika 4.3 Arhitektura višeklasnog modela	33
Slika 4.4 Arhitektura rekonstrukcijskog modela	34
Slika 4.5 Komponentni prikaz integracije GHA-AIOps pristupa u GitHub Actions radni tok	39
Slika 4.6 Dijagram aktivnosti detekcije, izrade izvještaja i slanja obavijesti.....	42
Slika 5.1 Eksperimentalni protokol evaluacije GHA-AIOps pristupa	48
Slika 5.2 Matrica zabune na umjetnom ispitnom skupu podataka	52
Slika 5.3 Matrica zabune na ispitnome skupu GHALogs	54
Slika 5.4 Usporedba F1-mjere po klasama na umjetno generiranom skupu i GHALogs podskupu.....	56
Slika 5.5 ROC krivulja rekonstrukcijskog modela.....	59
Slika B.1 Prikaz LSTM ćelije [6]	71
Slika C.1 Prikaz arhitekture transformera [23].....	73

Popis Tablica

Tablica 3.1 Preslikavanje promatranih tipova anomalija u zapise radnog toka	20
Tablica 3.2 Uloga korištenih skupova podataka.....	24
Tablica 4.1 Funkcionalni i nefunkcionalni zahtjevi GHA-AIOps pristupa.....	27
Tablica 4.2 Prikaz parametara učenih modela.....	35
Tablica 4.3 Glavne programske komponente GHA-AIOps pristupa	37
Tablica 5.1 Mjere uspješnosti višeklasnog modela na umjetno generiranom ispitnom skupu	52
Tablica 5.2 Mjere uspješnosti višeklasnog modela na ispitnome skupu podataka GHALogs	54
Tablica 5.3 Sažetak glavnih rezultata višeklasnog modela	57

Kazalo kratica i oznaka

Kratica	Engleski naziv	Hrvatski naziv
AIOps	<i>Artificial Intelligence for IT Operations</i>	umjetna inteligencija za IT operacije
AUC	<i>Area under curve</i>	površina ispod ROC krivulje
API	<i>Application Programming Interface</i>	programsko sučelje aplikacije
BERT	<i>Bidirectional encoder representations from transformers</i>	dvosmjernan koderska reprezentacija za transformere
CI/CD	<i>Continuous Integration and Continuous Delivery</i>	kontinuirana integracija i isporuka
DevOps	<i>Development and Operations</i>	razvoj i operacije
FPR	<i>False positive rate</i>	stopa lažnog alarma
GHA-AIOps	<i>GitHub Actions Artificial Intelligence for IT Operations</i>	umjetna inteligencija GitHub Akcija za IT operacije
GHALogs	<i>GitHub Actions Logs</i>	zapisi GitHub Akcija
LSTM	<i>long short-term memory</i>	dugotrajna kratkoročna memorija
RNN	Recurrent neural networks	povratne neuronske mreže
ROC	Receiver Operating Characteristic	radna karakteristika prijemnika
ReLU	<i>Rectified Linear Unit</i>	reflektirana linearna jedinica
SMTP	<i>Simple Mail Transfer Protocol</i>	jednostavan protokol za prijenos pošte
TPR	True positive rate	Stopa stvarnih pozitiva/odziv

Oznaka	Značenje
TP	broj stvarno pozitivnih primjera
TN	broj stvarno negativnih primjera
FP	broj lažno pozitivnih primjera
FN	broj lažno negativnih primjera
K	broj klasa, ili vektor ključa ovisno o formuli
N	broj elemenata vektora odnosno uzoraka, ovisno o formuli
y	stvarna vrijednost
$\hat{y}$	predviđena ili rekonstruirana vrijednost
W	matrica težine neuronske mreže
b	vektor pomaka neuronske mreže
Q	vektor upita
V	vektor vrijednosti

1. Uvod

Moderni razvojno-operativni procesi (engl. Development and Operations - DevOps) u velikoj se mjeri oslanjaju na automatizirane procese kontinuirane integracije i isporuke (engl. Continuous Integration and Continuous Delivery - CI/CD). Danas se koriste različiti alati koji pomažu u automatizaciji. Jedan takav alat su GitHub Akcije (engl. GitHub Actions), GitHubov sustav za automatizaciju koji omogućuje izgradnju, ispitivanje i implementaciju koda kroz definirane tokove rada (engl. workflows) [1]. GitHub Akcije se često koriste kod projekata s javno dostupnim izvornim kodom koji često održavaju svoj kod na GitHub-u.

Iako postojeći nadzorni alati prikupljaju znatne količine telemetrijskih podataka, u inženjerskoj praksi postoji vidljiva praznina u automatiziranoj, ranoj detekciji specifičnih obrazaca anomalija isključivo unutar okruženja GitHub Actions. Nakon što se primijeti greška u automatiziranom sustavu isporuke potrebno je ručno pregledavati zapise i metapodatke koji nastaju tijekom autonomnog pokretanja [2]. Uočeni inženjerski problem ne odnosi se samo na neuspješna pokretanja radnih tokova. U CI/CD cjevovodima mogu se pojaviti anomalije koje ne dovode odmah do prekida izvođenja, ali upućuju na povećan operativni rizik, degradaciju pouzdanosti ili pogoršanje održivosti razvojnog procesa. Primjeri takvih pojava uključuju konfiguracijski pomak, nestabilne automatske ispite, tihe neuspjehe i sigurnosno osjetljive zapise.

Zapisi (engl. logs) predstavljaju polustrukturirane tekstualne podatke generirane izvršavanjem isječaka izvornog koda. Provjera stabilnosti i robusnosti programskog sustava često se oslanja na analizu ovih zapisa, s obzirom na to da oni čine jedini trag o ponašanju sustava tijekom njegova rada. S radom sustava u produkcijskoj okolini, količina generiranih zapisa raste, što zahtijeva njihovo kontinuirano prikupljanje za analizu. Struktura pojedinog zapisa može se značajno razlikovati ovisno o vrsti događaja koju opisuje ili o stilu pisanja autora izvornog koda. Navedena varijabilnost posebno dolazi do izražaja na platformama poput GitHuba, gdje na razvoju istog softverskog projekta sudjeluje velik broj globalno raširenih korisnika. Veće količine zapisa usporavaju njihov ručni pregled. Sve brži ciklusi

razvoja i implementacije novih inačica programske podrške izravno utječu na promjene same strukture zapisa [5].

Zbog sve veće količine podataka generiranih tijekom razvojnih procesa, osobito u CI/CD sustavima poput GitHub Actions, za njihovu analizu sve se češće primjenjuju modeli strojnog učenja i dubokog učenja, uključujući velike jezične modele (engl. Large Language Models-LLM) [6], [7], [24]. Takvi pristupi omogućuju predviđanje vjerojatnosti pojave pogrešaka te učenje uobičajenog ponašanja sustava radi detekcije anomalija. Iako je moguće djelomično automatizirati otklanjanje grešaka, takve funkcionalnosti još uvijek nisu u potpunosti standardizirane te se većinom nalaze u fazi napredne primjene ili istraživačkog razvoja [8],[9].

U radu je oblikovan pristup za detekciju anomalija u zapisima GitHub Actions radnih tokova primjenom umjetne inteligencije za IT operacije, koji se radi sažetog označavanja u nastavku naziva GHA-AIOps (engl. GitHub Actions Artificial Intelligence for IT Operations). Pristup obuhvaća četiri vrste anomalija: sigurnosne anomalije, tihe neuspjehe, nestabilne ispite (engl. *flaky tests*) i konfiguracijski pomak [11], [12], [13], [14]. Zapisi se najprije prevode u numeričke reprezentacije s pomoću autoenkodera temeljenog na povratnim neuronskim mrežama i jednodimenzionalnoj konvoluciji [22]. Dobiveni vektori zatim se grupiraju kliznim prozorima i prosljeđuju dvama modelima transformera. Prvi model provodi višeklasnu klasifikaciju zapisa, dok drugi model uči uobičajenu raspodjelu zapisa te određuje anomalije na temelju rekonstrukcijske pogreške. Takva je kombinacija odabrana jer autoenkoder omogućuje dobivanje numeričkih reprezentacija nestrukturiranih zapisa bez oslanjanja na krute predloške zapisa, dok transformer omogućuje analizu konteksta unutar slijeda zapisa [23], [24].

Za učenje ovih modela razrađen je postupak izgradnje umjetnog skupa podataka za ispitivanje temeljen na simuliranju izgradnje i ispitivanja poslužiteljske aplikacije. Na temelju toga je generiran umjetni skup podataka s kontrolirano ubačenim anomalijama, uz dodatnu obradu podskupa stvarnih zapisa iz javnih repozitorija GHALogs [18]. Za integraciju modela u radni tok izrađena je javno dostupna GitHub Akcija koja omogućuje obradu zapisa unutar stvarnog radnog toka te automatsko slanje izvještaja elektroničkom poštom u slučaju detektiranih degradacija.

Predloženi pristup empirijski je evaluiran na umjetno generiranom označenom skupu podataka i na označenom podskupu stvarnih GHALogs zapisa. Za višeklasni

model korištene su metrike točnosti, preciznosti, odziva i F1-mjere, pri čemu su rezultati analizirani zasebno po tipu anomalije i agregirano makro prosjekom. Takav je način vrednovanja važan jer ukupan udio standardnih zapisa može prikriti slabiju detekciju rjeđih anomalijских kategorija [3]. Za rekonstrukcijski model korištena je ROC krivulja i površina ispod ROC krivulje jer se odluka o anomaliji temelji na promjenjivom pragu rekonstrukcijske pogreške [4]. Uspješnost pristupa promatra se kroz sposobnost razlikovanja standardnih i anomalijских zapisa, očuvanje detekcije rjeđih anomalijских kategorija te stabilnost modela pri primjeni na podatke koji se razlikuju od kontrolirano generiranog skupa. Provedena evaluacija na označenom podskupu GHALogs zapisa koristi se za procjenu primjenjivosti pristupa na heterogenije zapise iz javno dostupnih razvojnih procesa, pri čemu se rezultati tumače u okviru korištenih skupova podataka i provedenog eksperimentalnog protokola. Time se uspješnost predloženog pristupa ne vrednuje samo kroz sposobnost modela da nauči kontrolirano generirane obrasce, nego i kroz njegovu primjenjivost na heterogenije i manje pravilne zapise iz javno dostupnih razvojnih procesa.

U drugom poglavlju prikazuju se teorijska osnova i srodni pristupi potrebni za oblikovanje GHA-AIOps pristupa te se definira klasifikacija promatranih tipova anomalija u CI/CD zapisima. Poglavlje 3 opisuje podatkovni temelj GHA-AIOps pristupa, strukturu podataka GitHub Akcija, izgradnju umjetno generiranog skupa podataka i uporabu podskupa GHALogs te postupke označavanja, predobrade i oblikovanja značajki. U poglavlju 4 prikazana je implementacija GHA-AIOps pristupa, s naglaskom na autoenkoder, modele transformera i integraciju putem GitHub Akcije. U petom poglavlju prikazana je eksperimentalna evaluacija GHA-AIOps pristupa, uključujući metodologiju evaluacije, korištene mjere uspješnosti, dobivene rezultate te raspravu o njihovoj interpretaciji i ograničenjima. U zaključku su sažeti glavni doprinosi rada, ograničenja i smjernice za daljnji razvoj modela. Dodaci sadržavaju opis repozitorija, upute za reprodukciju rezultata i upute za pokretanje izvornog koda te dopunske tehničke opise LSTM ćelije i arhitekture transformera.

2. Pregled AIOps pristupa i anomalija u CI/CD zapisima

U ovom poglavlju dan je pregled generativnih i prediktivnih AIOps pristupa razvojno operativnim procesima relevantnih za ovaj rad: prediktivno, rekonstrukcijsko i generativno otkrivanje anomalija, formalizaciju kvarova i anomalija u CI/CD okruženju. Prikazane su osnove reprezentacije nestrukturiranih zapisa te primjena transformera u kontekstualnoj obradi slijeda zapisa. Definirani su zadaci integracije i isporuke programske podrške i anomalije koje se mogu pojaviti u njihovim zapisima.

Na temelju ovog pregleda u nastavku rada oblikuju se skup podataka, postupak izlučivanja značajki, modeli za detekciju anomalija i način integracije u GitHub Actions radni tok.

2.1. AIOps pristupi detekciji anomalija u razvojno-operativnim procesima

AIOps se u primjenjuje kroz različite alate koji se koriste za prikupljanje velike količine podataka, obrađivanje tih podataka, strojno učenje i nadziranje sustava u stvarnome vremenu. U razvojno-operativnim procesima posebno su važni pristupi koji obrađuju zapise, metapodatke i vremenski uređene sljedove događaja nastale tijekom automatizirane izgradnje, ispitivanja i isporuke programske podrške. Pri čemu se anomalije ne moraju pojaviti samo kao potpuni prekid izvođenja, nego i kao neuobičajeno trajanje, neočekivan slijed koraka, upozorenje, promjena konfiguracije ili sigurnosno osjetljiv ispis [15]. Pristupi detekciji u AIOps kontekstu mogu se razvrstati na prediktivne, rekonstrukcijske i generativne.

Prediktivni modeli koriste prethodno prikupljene podatke kako bi odredili buduće stanje sustava ili vjerojatnost da u sustavu dođe do pogreške, uskog grla i sigurnosnih rizika. Takvi modeli promatraju prethodni slijed događaja i pokušavaju predvidjeti očekivani nastavak slijeda. Primjerice, ako se u zapisima pojavljuje slijed koraka (A, B, C), model može pokušati predvidjeti sljedeći očekivani korak (D). Ako se stvarni nastavak značajno razlikuje od predviđenoga, takvo se odstupanje može

smatrati anomalijom. Modeli koji koriste označene podatke za nadzirano učenje se primjenjuju na klasičnim metodama strojnog učenja kao što su logistička regresija i stroj s potpornim vektorima (engl. support vector machine). Klasični pristupi strojnom učenju predviđaju vjerojatnost da će u sustavu doći do pogreške u nekom koraku. Prednost je mogućnost izravnog razvrstavanja zapisa u unaprijed definirane klase, dok je glavno ograničenje potreba za dovoljno kvalitetnim i reprezentativnim označenim skupom podataka.

Rekonstrukcijski modeli primjenjuju se kada je cilj naučiti uobičajeno ponašanje sustava i zatim detektirati odstupanja od tog ponašanja. Umjesto predviđanja sljedećeg događaja, rekonstrukcijski model prima cijeli ulazni prozor zapisa, primjerice slijed (A, B, C, D), pretvara ga u latentnu reprezentaciju i pokušava rekonstruirati izvorni ulaz. Ako prozor odgovara obrascima naučenima tijekom učenja, rekonstrukcijska pogreška bit će mala. Ako prozor sadrži anomalijski zapis ili neuobičajen slijed događaja, rekonstrukcijska pogreška obično raste, pa se anomalija određuje usporedbom pogreške s unaprijed zadanim ili adaptivno određenim pragom.

X. Zhang i sur. [6] primjenjuju model koji koristi kombinaciju LSTM neuronske mreže i varijacijskog autoenkodera (engl. Variational Autoencoder - VAE) za otkrivanje anomalija u zapisima. Model koristi LSTM za učenje slijednih ovisnosti u zapisima kako bi izvukao semantičku odnose u zapisima sustava. Varijacijski autoenkoder oblikuje latentnu reprezentaciju i omogućuje rekonstrukciju ulaza. Anomalija se određuje mjerenjem razlike između ulaza i izlaza modela. C. Hemalatha i V. K. Vatsavayi [7] primjenjuju model koji koristi kombinaciju transformera i varijacijskog autoenkodera. Ulazni zapisi prvo prolaze kroz koder transformera pa se korištenjem trika reparametrizacije pretvaraju u latentnu reprezentaciju koja se koristi za rekonstrukciju niza. U ovim pristupima prag rekonstrukcijske pogreške određuje granicu između standardnog i anomalijskog ponašanja. Koristi se adaptivni prag kako bi se odredila kolika pogreška u rekonstrukciji predstavlja anomaliju.

Rekonstrukcijski modeli u [6] i [7] oslanjaju se na učenje obrazaca zapisa i mjere odstupanje. Predlošci zapisa su ovisni o skupu podataka iz kojeg su izvučeni. Modeli trenirani na ovakav način nisu otporni na promijene u oblicima zapisa.

Generativni pristupi danas uobičajeno koriste velike jezične modele za stvaranje zapisa o problemu sustava. Generativni modeli usmjereni su na pregledavanje

zapisa koje stvara sustav te mogu ponuditi objašnjenje pogreške kada dođe do nje. Ovaj pristup može pomoći razvojnom timu u razumijevanju nastalog problema. Rani modeli su uglavnom izdavali upozorenja ili pokretali unaprijed određene postupke. Današnji modeli omogućavaju integraciju s platformama za timski rad kao što je Microsoft Teams. Veliki jezični modeli mogu iz prirodnog jezika generirati konfiguracijske datoteke radnog toka, automatske ispite i radna okruženja [8]. Veliki jezični modeli imaju mogućnost objasniti stanje sustava, otkriti anomalije i njihov uzrok te poduzeti postupke otklanjanja anomalije ili greške. W. Xu i sur. [9] su napravili radni okvir LogSage koji koristi veliki jezični model za pregledavanje i analizu zapisa grešaka te nudi objašnjenja s primjerima zapisa i prošlih grešaka. Ovakvi generativni pristupi nisu primarno usmjereni na strogu, ponovljivu i kvantitativno mjerljivu klasifikaciju više tipova anomalija u stvarnom CI/CD radnom toku. Dodatni izazov kod generativnog pristupa su halucinacije koje mogu nastati kod rijetkih anomalija ili grešaka. Stoga zahtijevaju provjeru generiranih objašnjenja i nisu zamjena za ponovljivu evaluaciju modela za detekciju anomalija.

Prediktivni modeli korisni su kada je cilj procijeniti očekivani nastavak slijeda događaja, a rekonstrukcijski modeli prikladni su kada se želi naučiti standardno ponašanje i detektirati odstupanja od tog ponašanja, dok su klasifikacijski modeli prikladni kada je potrebno razlikovati više unaprijed definiranih tipova anomalija.

Iz prikazanih ograničenja proizlazi da je za zadatak ovog rada potreban pristup koji ne ovisi isključivo o ručno definiranim predlošcima zapisa, nego može obraditi nestrukturirane retke zapisa, njihov slijedni kontekst i unaprijed definirane klase anomalija.

2.2. Tipovi anomalija u CI/CD zapisima

Integracija i isporuka programske podrške se sastoji od mnogo manjih poslova koji pokrivaju različita područja koja mogu stvarati različite anomalije. Anomalije ne znače da će nužno doći do pogreške ili zaustavljanja integracijskog cjevovoda. Mogu značiti da raspoloživi resursi nisu optimalno iskorišteni.

2.2.1. Kvarovi i degradacije u CI/CD cjevovodima

Moderni procesi kontinuirane integracije i isporuke (CI/CD) mogu biti izloženi čestim kvarovima i degradacijama učinkovitosti cjevovoda zbog rastuće složenosti sustava, paralelnog izvođenja zadataka i vanjskih ovisnosti. Prema istraživanju Simin Sun i sur. [10] zadaci integracije i isporuke se mogu podijeliti prema sjedećim kategorijama:

- **Kvaliteta koda** uključuje automatsko ispitivanje koda, na primjer s jediničnim ispitima (engl. unit tests) i statičnu analizu koda.
- **Sigurnost** predstavlja provjere sigurnosnih nedostataka u vlastitom kodu ili ako postoje poznati nedostaci stranih knjižnica ili radnih okvira koji se koriste u razvojnom procesu.
- **Ovisnost i usklađenost** provjerava usklađenost različitih verzija programske podrške međusobno i s različitim arhitekturama sustava na kojem se koristi. Upravljanje licencama osigurava da je sva strana programska podrška ispravno licencirana.
- **Infrastruktura i konfiguracija** osigurava da je programska podrška ispravno konfigurirana za namijenjenu infrastrukturu.
- **Postavljanje i puštanje programske potpore u upotrebu** uključuje simuliranje instalacije u okruženju za koje je namijenjena.

Kvarovi i degradacije performansi najčešće nastaju kao rezultat pogreške ili neusklađenosti u jednoj ili više navedenih kategorija. Empirijske analize kvarova u CI/CD procesima, ukazuju na to da pogreške pri izgradnji (engl. build failures) često proizlaze iz neusklađenih ovisnosti, loše konfiguracije i grešaka u samim ispitnim slučajevima [13]. Pritom je važno razlikovati kvarove koji korisno zaustavljaju cjevovod jer su otkrili stvarnu grešku u programskom kodu od kvarova i degradacija koje nepotrebno usporavaju razvojni proces ili smanjuju pouzdanost automatiziranih provjera [10].

Upozorenja o zastarjelim ovisnostima, preskočeni ispiti, promjene u konfiguraciji, sigurnosno osjetljivi ispisi ili nestabilni rezultati ispitivanja mogu ostati prikriveni unutar velike količine zapisa i metapodataka. Njihovo ručno pretraživanje povećava kognitivno opterećenje razvojnih inženjera i usporava razvojni ciklus. Zbog toga se

takvi obrasce u ovom radu ne promatraju samo kao izolirane tehničke pogreške pojedinih koraka, nego i kao pokazatelji operativnog rizika u razvojno-operativnom procesu. Takav rizik uključuje smanjenu pouzdanost automatiziranih provjera, povećano vrijeme ručne analize, mogućnost propuštanja stvarnih grešaka te sigurnosno osjetljive obrasce koji ostaju prikriveni u zapisima radnog toka.

Navedene kategorije ne predstavljaju konačan popis svih mogućih anomalija u CI/CD cjevovodima, nego širi prostor problema iz kojega se u ovom radu izdvajaju one anomalije koje se mogu očitovati u zapisima GitHub Actions radnih tokova i za koje je moguće oblikovati skup podataka za učenje i evaluaciju modela. U nastavku se zato navedeni operativni problemi svode na četiri klase koje se mogu prepoznati iz zapisa GitHub Actions radnih tokova: nestabilne ispite, konfiguracijski pomak, tihe neuspjehe i sigurnosne anomalije.

U ovom radu degradacije performansi promatraju se u mjeri u kojoj se očituju kroz zapise, upozorenja, promjene redoslijeda izvođenja, preskočene korake ili druge obrasce povezane s operativnim rizikom, a ne kao zasebna analiza vremenskih regresija svih radnih tokova.

2.2.2. Specifičnosti GitHub Actions radnih tokova i odabrane klase anomalija

GitHub Actions okruženje ima hijerarhijsku strukturu koja utječe na način na koji se anomalije pojavljuju u zapisima. Radni tok sastoji se od jednog ili više poslova, a svaki posao od niza koraka.

U ovom radu promatraju se četiri tipa anomalija: nestabilni ispiti, konfiguracijski pomak, tihi neuspjesi i sigurnosne anomalije. Te su klase odabrane zato što pokrivaju različite izvore rizika u CI/CD cjevovodu: kvalitetu programskog koda, stabilnost ispitivanja, usklađenost konfiguracije i ovisnosti, pouzdanost automatiziranog izvođenja te sigurnost zapisa i radnog okruženja. Prednost je da se navedene anomalije mogu očitovati u zapisima radnog toka, što ih čini prikladnima za modelsku detekciju na temelju GitHub Actions zapisa.

Anomalije mogu nastati kao rezultat greške u jednoj ili više od navedenih kategorija zadataka. U nastavku su navedeni tipovi anomalija i zadaci na koje se odnose.

Nestabilni ispiti: odnosi se na kvalitetu koda i automatsko ispitivanje. Kada rezultati ispitivanja nisu jednaki za istu verziju programske podrške nazivaju se nestabilni [11]. Ovakve rezultate ispitivanja može biti teško reproducirati jer se mijenjaju kod svakog pokretanja ispitivanja. Mogu utjecati na vrijeme koje je potrebno da se odstrani greška tako što lažno prikazuju pad ispita koji nije izazvan nedavnim promjenama u kodu. Ovakvi se ispiti često ignoriraju što dovodi do toga da se ponekad preskoče i stvarne greške. Glavni razlozi loših ispitivanja su: pozivi asinkronih funkcija, istovremeno pokretanje više dretvi, curenje resursa, ovisnost o mreži, ovisnost o točnosti mjerenja vremena, ulazno-izlazne operacije, generiranje nasumičnih brojeva, korištenje brojeva s lebdećom točkom i nepoređanih skupova.

Konfiguracijski pomak (engl. configuration drift): odnosi se na infrastrukturu, konfiguraciju, ovisnost, usklađenost, postavljanja i puštanja u upotrebu. Kod isporuke programske podrške potrebno je pravilno postaviti okruženje u kojem se ona pokreće. Kada se stvarno stanje tog okruženja nenamjerno razlikuje od potrebnog to se naziva konfiguracijski pomak . Moguće posljedice konfiguracijskog pomaka su smanjenje učinkovitosti, sigurnosni nedostaci i smanjena stabilnost. Što je sustav veći to je potrebno utrošiti više vremena kako bi se osigurala ispravna konfiguracija okruženja. Promjene u kodu mogu ponekad zahtijevati promjene konfiguracije što dovodi do konfiguracijskog pomaka. Ako knjižnice ili radni okviri dobiju nove verzije potrebno je tome prilagoditi konfiguraciju okruženja kako bi izbjegli konfiguracijski pomak i sigurnosne nedostatke starijih verzija.

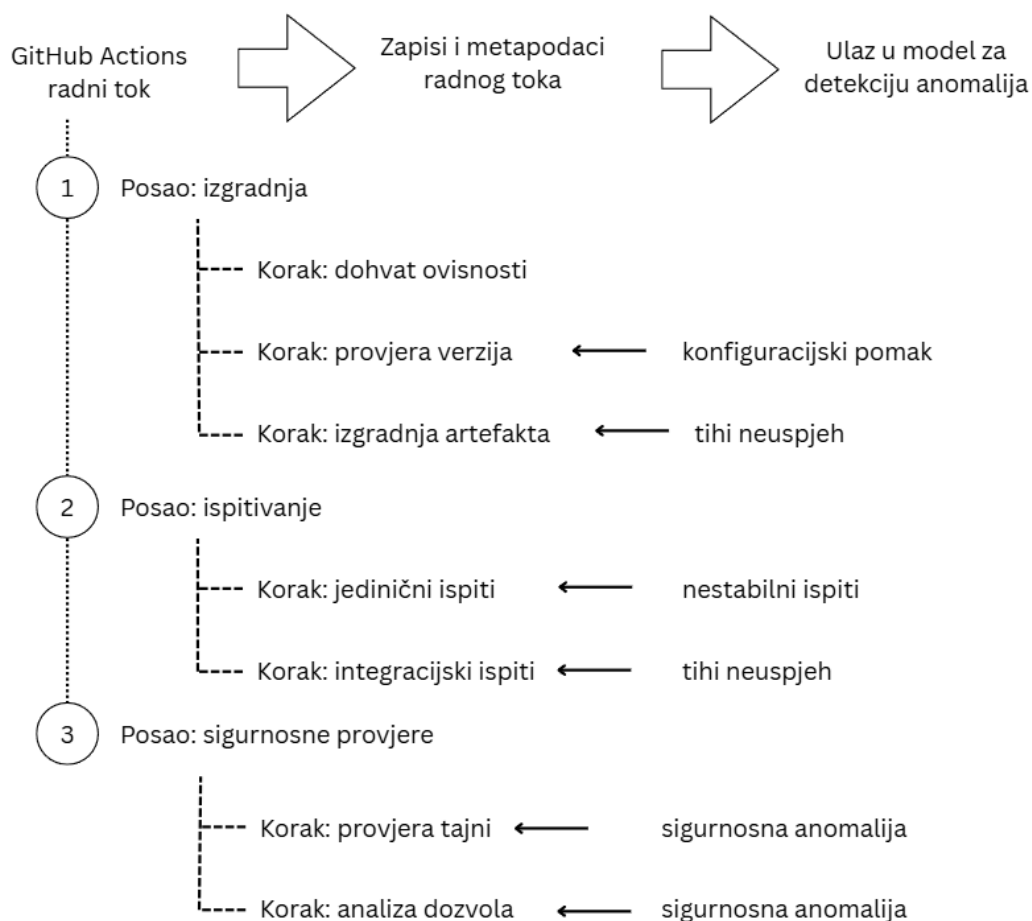
Tihi neuspjesi (engl. silent failures): može se odnositi na bilo koju od navedenih kategorija poslova integracije. Kada integracijski posao prikazuje uspješni završetak, ali u stvarnosti je barem djelomično neuspješan u namijenjenoj zadaći naziva se tihi neuspjeh [13]. Ovakvi neuspjesi se često ne pronađu dok ne dođe do veće greške. Rezultat navedenih neuspjeha je smanjenje automatizacije radnih tokova i usporavanje odstranjivanja pogrešaka. Neki od uzroka tih neuspjeha su problemi s priručnom memorijom, upravljanje artefaktima integracije, ignorirana automatska ispitivanja i sigurnosne pretrage, problemi s infrastrukturom, nepravilno korištenje integracijskih alata, neviđena upozorenja i neprimijećeni problemi kod puštanja programske podrške u upotrebu.

Sigurnosne anomalije: obuhvaćaju zapise i obrasce koji mogu upućivati na sigurnosni rizik u CI/CD procesu. Česti oblik ovakvih anomalija su tvrdo kodirane

tajne vrijednosti, nesigurne varijable okruženja i nepravilno određeni tokeni. Ovi nedostaci su prisutni u više od četvrtine radnih tokova na GitHub Akcijama [14]. Većina radnih tokova je nepravilno konfigurirana, sadrže greške u YAML datotekama i loše izolirane poslove u zastarjelim Akcijama.

Navedene klase ne obuhvaćaju sve moguće anomalije u CI/CD okruženju, nego predstavljaju skup anomalija koje su u ovom radu odabrane zbog povezanosti sa zapisima radnog toka i mogućnosti označavanja u korištenim skupovima podataka.

Slika 2.1 prikazuje moguće mjesto pojavljivanja anomalija u hijerarhiji GitHub Actions radnog toka. Radni tok sastoji se od poslova i koraka, a svaki korak tijekom izvođenja generira zapise koji se mogu koristiti kao ulazni podaci za detekciju anomalija. Slika konceptualno povezuje strukturu radnog toka s četirima klasama anomalija promatranima u ovom radu: nestabilnim ispitima, konfiguracijskim pomakom, tihim neuspjesima i sigurnosnim anomalijama.



Slika 2.1 Konceptualni prikaz mogućih mjesta pojave promatranih anomalija u GitHub Actions radnom toku.

Na temelju prikazanog pregleda može se zaključiti da detekcija anomalija u GitHub Actions zapisima zahtijeva pristup koji istodobno uzima u obzir tekstualni sadržaj zapisa, njihov slijedni kontekst i različite oblike operativnog rizika. Prediktivni i rekonstrukcijski pristupi prikladni su za učenje uobičajenog ponašanja sustava i otkrivanje odstupanja, dok je višeklasna klasifikacija potrebna kada se anomalije žele razvrstati u unaprijed definirane kategorije. Budući da se navedene pojave očituju u nestrukturiranim zapisima radnih tokova, sljedeće poglavlje opisuje izvore podataka, postupak preslikavanja tih anomalija u konkretne tragove u zapisima te oblikovanje skupa podataka za učenje i ispitivanje modela.

3. Podatkovna osnova GHA-AIOps pristupa

U ovom poglavlju opisan je podatkovni temelj predloženog pristupa za detekciju anomalija u GitHub Actions radnim tokovima. Prikazani su izvori zapisa GitHub Actions radnih tokova, zatim se definiraju načini kojima se promatrane anomalije očituju u zapisima, postupci označavanja umjetno generiranih i stvarnih zapisa, predobrada tekstualnih zapisa, oblikovanje značajki te podjela podataka na skupove za učenje, prilagodbu i ispitivanje.

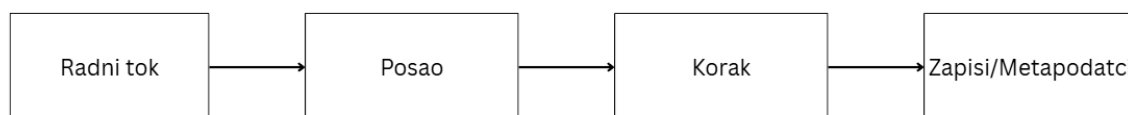
3.1. Ulazni podaci i izvori zapisa

Ovo potpoglavlje objedinjuje opis GitHub Actions podataka, stvarnog skupa GHALogs i umjetno generiranog skupa podataka, čime se definira izvor podataka te njihova uloga u razvoju i evaluaciji sustava. Umjetno generirani skup koristi se za kontrolirano učenje i početno ispitivanje modela, dok se podskup GHALogs koristi za ograničenu provjeru primjenjivosti na stvarnim, heterogenijim zapisima, a ne kao dokaz opće robusnosti na svim GitHub Actions radnim tokovima.

3.1.1. Struktura zapisa GitHub Akcija i metapodataka

GitHub Akcije se sastoje od radnih tokova, poslova i koraka. Radni tokovi se sastoje od skupa poslova i započinju odvijanje kada se ispune zadani uvjeti definirani u YAML datoteci. Ti uvjeti mogu biti promjene koda, dodavanje nove oznake, spajanje grana itd. Poslovi se odvijaju paralelno, svaki u svojem okruženju osim ako nije drugačije definirano. Ovisno o konfiguraciji radnog toka, neuspjeh jednog posla može utjecati na izvođenje drugih poslova, osobito ako postoje definirane ovisnosti.

Tijekom izvođenja koraka nastaju zapisi (engl. *logs*), odnosno redci standardnog izlaza alata, skripti ili akcija koje se izvode u radnom toku. Hijerarhijska struktura GitHub Actions okruženja i odnos između izvršnih cjelina te generiranih zapisa i metapodataka grafički su prikazani na slici 3.1.



Slika 3.1 Hijerarhija izvođenja i generiranja podataka unutar GitHub Actions radnog toka

Posao se sastoji od koraka koji tijekom rada stvaraju zapise na standardni izlaz. Svakom redu zapisa dodaje se vremenska oznaka (engl. *timestamp*) kada je nastao. Osim vremenske oznake ovi zapisi su nestrukturirani. Kako bi se koristili za treniranje modela često ih je potrebno unaprijed obraditi koristeći regularne izraze ili parsere zapisa kao što je Drain [17]. Na razini radnog toka i posla mogu se pratiti dodatni metapodaci, kao što su status izvođenja, trajanje, vrijeme čekanja i identifikatori potrebni za dohvat podataka preko GitHub API-ja. GitHub također prikazuje vrijeme potrebno za obavljanje radnih tokova, vrijeme čekanja da oni započnu i omjere njihovih uspjeha i neuspjeha. GitHub pruža mogućnost korištenja vlastitih servera za pokretanje radnih tokova. Ovako pokrenuti tokovi mogu pružati dodatne podatke koji nisu dostupni samo s GitHuba kao što su opterećenje memorije, procesora i mreže. Moguće je i prikupiti zapise okruženja u kojem se odvija posao. Ovi podaci mogu biti korisni kod određivanja anomalije na razini posla. Ovdje je bitno spomenuti da na platformi GitHub postoji LLM kopilot koji može pojasniti pogreške u zaustavljenim radnim tokovima, ali nema mogućnost detekcije anomalija koje ne zaustavljaju radni tok [16].

Završni status radnog toka sam po sebi nije dovoljan za prepoznavanje svih relevantnih problema jer se anomalija može očitovati kao upozorenje, neočekivan ispis, promjena redoslijeda izvođenja, neuobičajeno trajanje ili sigurnosno osjetljiv sadržaj unutar zapisa. Stoga se ne promatra isključivo binarni uspjeh radnog toka, već slijed zapisa nastao tijekom izvođenja posla. Budući da su zapisi nestrukturirani i njihov oblik ovisi o konkretnom alatu koji ih generira, redci zapisa nemaju jedinstvenu strukturu, zbog čega ih je prije uporabe u modelima nužno preobraditi i prevesti u numerički oblik.

3.1.2. Umjetno generirani skup podataka

Za potrebe ovog rada izrađena je programska podrška koja generira zapise. Za generiranje podataka korišten je model poslužiteljske aplikacije koja se spaja na bazu podataka. Programska podrška simulira njezinu izgradnju i automatsko ispitivanje. U zapise se ubacuju anomalije s njihovim oznakama koje služe za računanje mjera uspješnosti. Simulirana aplikacija predstavlja poslužitelja napravljenog u TypeScriptu koristeći paket express.

Umjetno generirani skup nužan je jer omogućuje kontrolirano umetanje anomalija i dobivanje s potpuno točnim i poznatim oznakama. Ovakav skup pruža uvjete za početno učenje i ispitivanje modela u izoliranom okruženju u kojem je strogo poznato koji su zapisi standardni, a koji anomalijski. U radnom toku korišteni su koraci izgradnje i automatskog ispitivanja, a pokretanja su izvođena i ručno i automatski. Korištena je strategija matrice (engl. *matrix strategy*) kojom se isti radni tok pokretao s različitim vrijednostima, a zapisi su potom preuzeti korištenjem GitHub REST API-ja.

Konfiguriran je radni tok s koracima izgradnje i automatskog ispitivanja, postavljeno je automatsko pokretanje radnog tok u zadanim vremenskim razmacima te je omogućeno i ručno pokretanje. Korištena je strategija matrice kod pokretanja toka što znači da se istovremeno odvija isti radni tok više puta s mogućim različitim vrijednostima. Svaki različiti slučaj dodaje samo jedan tip anomalije u zapise, osim prvog slučaja gdje se pokreće radni tok bez anomalija. Radni tok je ukupno pokrenut dvadeset puta. Zapisi su preuzeti korištenjem GitHub REST API-a. Dvije trećine pokretanja su stavljene u skup za treniranje, a jedna trećina u ispitni skup.

3.1.3. Skup podataka GHALogs

U radu je upotrijebljen javno dostupni skup zapisa nastalih iz GitHub Akcija [18]. Ovaj skup podataka nastao je od preko dvjesto tisuća repozitorija s ukupno preko petsto tisuća pokretanja radnih tokova. U obzir su uzeti samo dobro ocijenjeni repozitoriji koji sadrže više od trideset pokretanja radnih tokova zadnjih devedeset dana prikupljanja zapisa 2023. godine. Ovaj je skup relevantan za rad jer sadrži zapise iz stvarnih razvojnih procesa, različitih projekata i heterogenih konfiguracija radnih tokova.

GHALogs u komprimiranom stanju zauzimaju preko 140 GB memorije, a ne komprimiranom zauzima preko 600 GB. Zapisi u skupu GHALogs su preuzeti iz javno dostupnih repozitorija bez dodatne obrade i označavanja. Za ispitivanje modela nije potreban cijeli skup podataka. Preuzet je podskup veličine 1 GB koji sadrži oko 1000 pokretanja radnih tokova.

3.2. Preslikavanje anomalija na zapise i označavanje zapisa

U poglavlju 2.2.2 definirane su četiri vrste anomalija promatrane u ovom radu: nestabilni ispiti, konfiguracijski pomak, tihi neuspjesi i sigurnosne anomalije. Za potrebe učenja i evaluacije modela, te apstraktne koncepte potrebno je prevesti u konkretne podatkovne tragove u zapisima. U nastavku se opisuje preslikavanje svake anomalije te postupci automatskog i ručnog označavanja.

3.2.1. Preslikavanje tipova anomalija na zapise radnog toka

Preslikavanje anomalija odnosi se na utvrđivanje konkretnih obrazaca u tekstualnim zapisima koji se smatraju pojavom određene klase. Svaka od četiri promatrane klase ima specifičan uzrok i specifičan način manifestacije u zapisima.

Nestabilni ispiti očituju se kao promjenjivi ili nedosljedni ishodi ispitivanja za istu verziju koda. U umjetno generiranom skupu takve su pojave ostvarene nasumičnim odabirom povratnih vrijednosti ispitivanih metoda, gdje jedna vrijednost predstavlja očekivani rezultat, a ostale pogrešne ishode.

Konfiguracijski pomak u zapisima se manifestira kroz odstupanje okruženja od očekivanog okruženja. U umjetno generiranom skupu tijekom provjere verzije nasumično se odabire hoće li se verzija promijeniti i za koliko. Promjene verzije su podijeljene u velike (engl. major), manje (engl. minor) i zakrpe (engl. patch).

Sigurnosne anomalije predstavljaju zapise i obrasce koji upućuju na sigurnosni rizik. U generiranim zapisima se prikazuju kroz zaostale ispise objekata koje sadrže korisničke informacije, tajne vezane uz pristup bazi podataka i vanjskim autentifikacijskim servisima. Osjetljivi podaci su generirani pri svakom pokretanju u obliku nasumičnog niza slova. Nasumično se odabire koje će se vrijednosti ispisati.

Tihi neuspjesi se događaju kada radni tok završi uspješno, ali nisu ispunjene sve njegove zadaće. U generiranim podacima se tihi neuspjesi prikazuju u obliku neispravnih licenci, nestalih artefakata i preskočenih ispita. Za neispravne licence se ispisuje upozorenje na koji se paket odnosi. Kod nestalih artefakata ispisuju se datoteke koje nestaju. Svakom preskočenom ispitu je naveden razlog preskakanja.

Sažeti pregled preslikavanja apstraktnih anomalija u konkretne podatkovne tragove prikazan je u Tablici 3.1.

Tablica 3.1 Preslikavanje promatranih tipova anomalija u zapise radnog toka

Tip anomalije	Podatkovni trag u zapisima	Primjer implementacije u radu
Nestabilni ispiti	Promjenjivi ishodi ispitivanja, povremeni padovi, različite povratne vrijednosti	<pre># neispravna povratna vrijednost AssertionError: {'status': 200, 'text': 'deletion successful'} != {'status': 204} - {'status': 200, 'text': 'deletion successful'} + {'status': 204}</pre>
Konfiguracijski pomak	Odstupanje od očekivanog stanja okruženja, promjena verzije paketa na više razina	<pre># provjera inačice paketa WARN Dependency configuration has changed WARN Package pg has new version ^8.16.3 -> ^8.16.6 WARN Patch update detected. No immediate risk</pre>
Sigurnosne Anomalije	Sigurnosni rizik, ispisi osjetljivih vrijednosti	<pre># ispis json objekta s korisničkim podacima [debug] user { username : aegilops password : 1fec45eae2a9bfa8ef6d08b22a role : student }</pre>
Tihi neuspjeh	Neizvršene zadaće, neispravne licence, nestali artefakti i preskočeni ispiti.	<pre># greška zbog nestale datoteke WARN Testing Method PUT terminated ERROR test.py :73:15: fatal error occurred: pom/test_values.json no such file or directory ERROR with open('pom/test_values.json', 'r') as file:</pre>

3.2.2. Označavanje umjetno generiranih zapisa

U umjetno generiranom skupu podataka oznake su dodjeljivane potpuno automatski. Pri ispisu svake linije koja je dio anomalije na početku se stavlja oznaka anomalije. Nestabilni ispiti se označavaju s <FLAKY>, konfiguracijski pomak s <DRIFT>, tihi neuspjesi sa <SILENT> i sigurnosne anomalije sa <SECURITY>. Svi zapisi koji nemaju oznake smatraju se standardnim.

3.2.3. Označavanje stvarnih GHALogs zapisa

Za razliku od simuliranih zapisa, podskup stvarnih GHALogs zapisa zahtijevao je naknadno ručno označavanje. Iz preuzetog skupa su izbačeni radni tokovi koji nisu imali više od jednog pokretanja. Za označavanje anomalija u zapisima su tražene ključne riječi „warn“, „skip“, „pass“, „fail“, „deprecated“, „token“, „password“, „secret“, „version“ i „licence“. Upozorenja za istekle licence, nestale datoteke i funkcije koje će uskoro postati zastarjele su se označavale kao tihi neuspjesi. Zastarjele verzije paketa, knjižnica i funkcije koje su već zastarjele su označene kao konfiguracijski pomak. Automatski ispiti koji su kroz više pokretanja pokazali različite rezultate su označeni kao nestabilan ispit. U preuzetome podskupu nisu pronađene sigurnosne anomalije niti je pronađeno curenje tajni ili zaporki.

3.3. Predobrada zapisa, izgradnja značajki i podjela podataka

Nakon faza prikupljanja i označavanja, nestrukturirane tekstualne zapise potrebno je očistiti, odvojiti od oznaka, prevesti u numeričke reprezentacije te segmentirati u sljedove prilagođene arhitekturi dubokih neuronskih mreža.

3.3.1. Predobrada tekstualnih zapisa

Kako bi se zapisi mogli koristiti u metodama dubokog učenja uklanjaju im se vremenske oznake. Iz zapisa se uklanjaju i oznake anomalija. Oznake anomalija se koriste za učenje i računanje mjera uspješnosti, ali se uklanjaju da ih modeli ne vide na ulazu. Zapis je prvo podijeljen na znakove te je svakome znaku dodijeljen broj. Riječ „Image“ bi postala niz 45, 81, 69, 75, 73. Brojeve su dobili svi znakovi ASCII

tablice između 32 i 127 osim njih posebno je dodijeljen broj za nepoznate znakove, znak \n i za nadopunjavanje niza.

3.3.2. Pretvaranje zapisa u vektore značajki

Tradicionalni pristupi pretvaranja zapisa u vektore značajki oslanjaju se na parsiranje. U ovom radu umjesto parsiranja zapisa u predloške kojima se dodjeljuju numeričke vrijednosti koristi se autoenkoder. Korišten je autoenkoder zbog otpornosti na promijene oblika zapisa i grupiranje semantički sličnih zapisa u slične vektore.

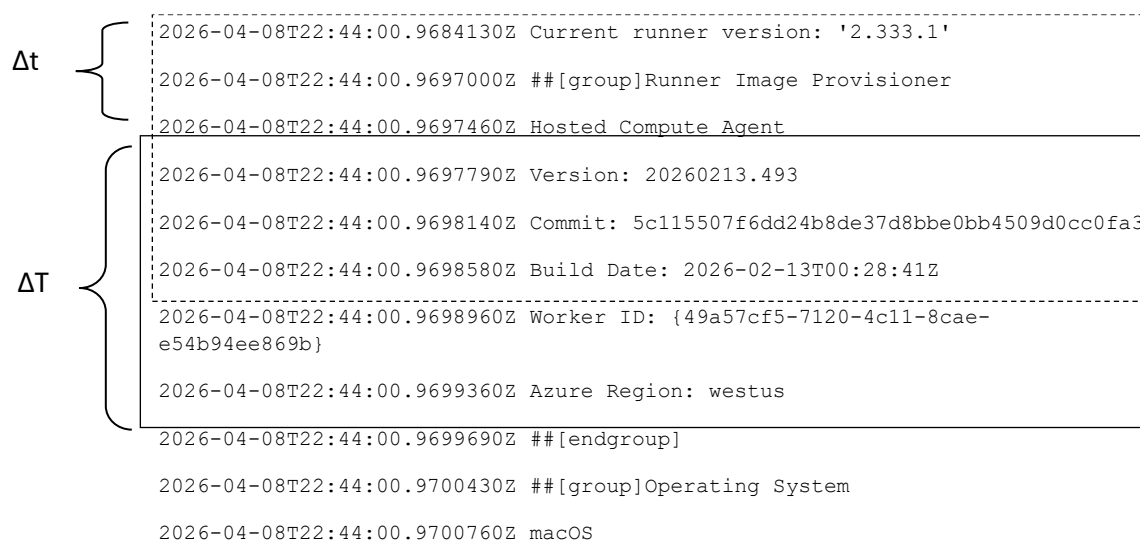
Zapisi GitHub Akcija se sastoje od standardnog ispisa poslova. Osim vremenske oznake nastajanja struktura ispisa ovisi o sadržaju akcije. Parsiranje razdvaja zapise na stalne i promjenjive dijelove kako bi se svrstali u predloške. Zapis „Downloading attrs-26.1.0-py3-none-any.whl (67 kB)“ bi parsiranjem postao „Downloading <*>“ gdje je „Downloading“ stalan dio uzorka, a „attrs-26.1.0-py3-none-any.whl (67 kB)“ promjenjiva varijabla. Parseri dodaju i oznake uzorka kao što je „event 1“. Postoje dvije vrste parsera heuristički i grupirajući. Kod grupirajućih parsera se računa udaljenost između zapisa koja se koristi za grupiranje zapisa u skupine iz kojih nastaju predlošci. Heuristički pristup prebraja broj ponavljanja riječi na pojedinoj poziciji kako bi stvorio potencijalne kandidate za stalan dio zapisa. Konačni predlošci se biraju među kandidatima [20].

Prema istraživanju Murphy A. i sur. [21] dva glavna nedostatka parsiranja su svrstavanje semantički sličnih zapisa u različite uzorke što može dovesti do lažnih uzbuna i nemogućnost prilagodbe promjeni oblika zapisa. Murphy A. i sur. su predložili radni okvir koji koristi leksičku bazu podataka engleskih riječi WORDNET i velike jezične modele za određivanje numeričkih reprezentacija zapisa.

3.3.3. Oblikovanje kliznih prozora

Izvlačenje značajki pridružuje zapisima numeričke vrijednosti koje se mogu koristiti algoritmima strojnog i dubokog učenja. Zapise je potrebno grupirati. U ovome radu zapisi su grupirani kliznim prozorom. Metoda kliznog prozora (engl. *sliding window*) je metoda podjele vremenskih podataka koja sadrži dva glavna parametra: veličinu prozora ΔT i veličinu pomaka Δt . ΔT predstavlja vremenski interval koji ulazi u prozor

kod zapisa tj. predstavlja broj zapisa koji ulaze u prozor. Δt predstavlja vremenski interval pomicanja prozora, a to je broj zapisa za koje se pomiče. ΔT i Δt ne trebaju biti jednaki, poželjno je da Δt bude manji od ΔT kako bi nastali preklapajući prozori (engl. *overlapping window*). Prikaz preklapajućih kliznih prozora nalazi se na slici 3.1. Na slici je isprekidanom crtom označen prijašnji prozor, Δt označuje pomak prozora. Punom crtom je označen trenutačni prozor veličine ΔT .



Slika 3.2 Prikaz kliznog prozora

3.3.4. Podjela podataka na skupove za učenje, prilagodbu i ispitivanje

Umjetno generirani zapisi su nakon preuzimanja korištenjem GitHub REST API-a raspodijeljeni u skupove za učenje i ispitivanje modela. Jednim pokretanjem radnog toka nastaju zapisi za svaki tip anomalije naveden u poglavlju 2. Prvih 14 takvih pokretanja je stavljeno u skup za učenje, a ostalih 6 je stavljeno u skup za ispitivanje.

Zapisi iz skupa GHALogs su podijeljeni u skupove za dodatno učenje (prilagodbu) i ispitivanje. Od svakog radnog toka u kojem se pojavljuju anomalije jedno pokretanje je svrstano u skup za dodatno treniranje i jedno pokretanje u ispitni skup.

Različita pokretanja istog radnog toka sadrže iste ili slične anomalije. Nisu iskorištena sva pokretanja jednog radnog toka da se izbjegne prenaučenos modela. Anomalije su slične u ispitnom skupu podataka i skupu za treniranje što znači da postoji rizik da rezultati budu bolji zbog sličnosti. U praksi bi se taj rizik dodatno

smanjio strožom podjelom na razini repozitorija ili radnog toka, pri čemu se pokretanja istog radnog toka ne bi pojavljivala u oba skupa.

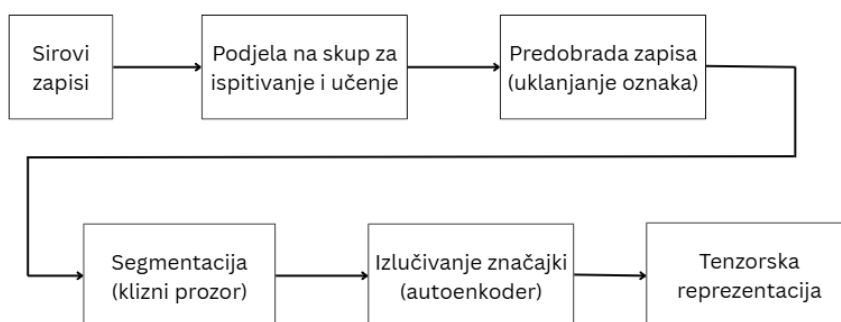
Pronađeno je devet radnih tokova s anomalijama. Stoga se u skupu za prilagodbu nalazi devet pokretanja radnih tokova, a u ispitnom skupu također devet pokretanja. Tablica 3.2 daje sažeti prikaz uloga svih analiziranih skupova.

Tablica 3.2 Uloga korištenih skupova podataka

Skup podataka	Izvor	Način označavanja	Uloga u radu
Umjetno generirani skup	Simulirani GitHub Actions radni tok	Automatsko označavanje pri ubacivanju anomalija	Početno učenje i bazno ispitivanje modela
Podskup GHALogs za prilagodbu	Stvarni GitHub Actions zapisi	Ručni pregled i označavanje ključnim riječima	Prilagodba stvarnim zapisima
Ispitni podskup GHALogs	Stvarni GitHub Actions zapisi	Ručni pregled i označavanje ključnim riječima	Provjera primjenjivosti više-klasnog modela na označenim stvarnim zapisima

3.4. Tenzorska reprezentacija podataka i neravnoteža klasa

Rezultat svih opisanih faza prikupljanja, označavanja, predobrade i segmentacije jest skup značajki pripremljen za učenje i ispitivanje modela. Cjelokupni cjevovod transformacije sirovih nestrukturiranih zapisa u numeričke oblike prikladne za strojno učenje prikazan je na slici 3.2.



Slika 3.2. Podatkovni cjevovod od sirovih zapisa do tenzorskih reprezentacija

U konačnom obliku podaci su reprezentirani kao tenzori koji sadržavaju sljedove vektora oblikovanih unutar kliznih prozora. Svaki se zapis kodira u vektor fiksne duljine, nakon čega se tako dobiveni vektori grupiraju u ulazne sljedove prikladne za učenje i ispitivanje modela. Ovisno o korištenoj arhitekturi, ovakav ulaz služi za višeklasno razvrstavanje zapisa ili za procjenu odstupanja rekonstruiranog slijeda od ulaznog slijeda.

Metodološki je važno istaknuti da oba korištena skupa podataka karakterizira izražena neravnoteža klasa (engl. class imbalance). U CI/CD cjevovodima velika većina zapisa pripada standardnom, očekivanom ponašanju, dok su anomalijski zapisi znatno rjeđi. Iako je takva raspodjela očekivana u stvarnim radnim tokovima, ona otežava vrednovanje modela jer visoka ukupna točnost (engl. accuracy) može prikriti slabiju sposobnost prepoznavanja rijetkih, ali operativno važnih anomalija.

Zbog toga se uspješnost modela u radu ne procjenjuje isključivo ukupnom točnošću, nego i mjerama koje opisuju. U fazi evaluacije nužno je razmatranje mjera koje opisuju ponašanje modela na manjinskim klasama: preciznosti, odzivom i F1-mjerom po pojedinim tipovima anomalija, njihovim makro prosjekom te matricom zabune. Ovakav pristup omogućuje vjerodostojniju procjenu uspješnosti modela u uvjetima neravnoteže klasa, jer sprječava da visoka uspješnost na većinskoj standardnoj klasi prikrije slabiju detekciju rijetkih, ali operativno važnih anomalijских klasa.

4. Implementacija GHA-AIOps pristupa za detekciju anomalija

U ovom poglavlju opisana je implementacija GHA-AIOps pristupa za detekciju anomalija u GitHub Actions zapisima. Najprije se prikazuju tok obrade i zahtjevi koje rješenje mora zadovoljiti. Zatim se opisuje autoenkoder kojim se zapisi prevode u vektorske reprezentacije te dva modela transformera: višeklasni model za razvrstavanje promatranih tipova anomalija i rekonstrukcijski model za otkrivanje odstupanja od naučenog standardnog ponašanja. Na kraju se prikazuje integracija razvijenih modela u GitHub Actions radni tok te se navode glavna ograničenja izvedbe.

4.1. Zahtjevi i prikaz implementiranog rješenja

Ulazne podatke za implementirani GHA-AIOps pristup čine tekstualni zapisi GitHub Actions posla koji su nakon predobrade prevedeni u numerički oblik i organizirani u klizne prozore. Takav oblik reprezentacije omogućuje promatranje pojedinačnih redaka zapisa ne samo izolirano, nego i u širem kontekstu susjednih događaja. Navedeno je ključno jer se određeni tipovi anomalija mogu prepoznati iz pojedinačnog zapisa (npr. ispis sigurnosno osjetljive vrijednosti), dok se druge očituju isključivo u specifičnom slijedu zapisa ili neuobičajenoj kombinaciji poruka.

Arhitektura implementiranog rješenja prati strogi tok obrade: dohvat zapisa, predobrada i kodiranje, oblikovanje kliznih prozora, primjena modela za detekciju anomalija te izrada izvještaja uz slanje obavijesti. Za pretvaranje pojedinačnih redaka zapisa u vektorske reprezentacije koristi se autoenkoder. Tako dobiveni vektori grupiraju se u prozore i predaju modelima transformera. Višeklasni model transformera ima zadaću određivanja pripada li pojedini zapis standardnoj klasi ili jednoj od promatranih anomalijских klasa. S druge strane, rekonstrukcijski model transformer ne određuje specifičan tip anomalije, već pokušava rekonstruirati ulazni slijed, pri čemu se odstupanje (anomalija) detektira na temelju visoke rekonstrukcijske pogreške.

Kako bi se jednoznačno odredio i vrednovao opseg implementacije, GHA-AIOps pristup formaliziran je kroz skup funkcionalnih i nefunkcionalnih zahtjeva. Funkcionalni zahtjevi opisuju što rješenje mora izvesti nad GitHub Actions zapisima, dok nefunkcionalni zahtjevi određuju važna svojstva rješenja, uključujući primjenjivost na nestrukturirane zapise, smanjenu ovisnost o ručno definiranim predlošcima, otpornost na promjene oblika zapisa, provjerljivost na stvarnim zapisima i mogućnost uključivanja u CI/CD okruženje. Sažeti pregled postavljenih zahtjeva i načina njihovog ostvarenja u radu sistematiziran je u Tablici 4.1.

Tablica 4.1 Funkcionalni i nefunkcionalni zahtjevi GHA-AIOps pristupa

Oznaka	Zahtjev	Ostvarenje
FZ-1	Dohvat zapisa odabranog GitHub Actions posla.	Dohvat zapisa ostvaruje se namjenskom GitHub Akcijom i Python skriptom uporabom službenog GitHub API-ja.
FZ-2	Pretvorba redaka zapisa u numeričke reprezentacije prikladne za modele dubokog učenja.	Pretvorba redaka zapisa u numeričke reprezentacije ostvaruje se dodjelom cijelog broja svakom znaku zapisa. Koder autoenkodera prevara niz znakova u vektore jednake duljine.
FZ-3	Oblikovanje sljedova zapisa prikladnih za obradu modelima transformera.	Za oblikovanje sljedova prikladnih za obradu modelima transformera korišten je klizni prozor.
FZ-4	Detekcija anomaljskih zapisa i razlikovanje promatranih tipova anomalija kod višeklasnog modela.	Detekcija i razlikovanje anomaljskih zapisa ostvarena je primjenom normalizirajuće eksponencijalne aktivacijske funkcije (softmax) na izlaze višeklasnog modela čime se računa vjerojatnost pripadanja nekom tipu anomalije.
FZ-5	Izrada izvještaja o detektiranim anomalijama i obavješćavanje korisnika.	Izrada izvještaja o detektiranim anomalijama je ostvarena namjenskom GitHub Akcijom i Python skriptom koja predaje zapise učenom višeklasnom modelu. Obavješćavanje korisnika izvodi se slanjem e-pošte koristeći GitHub Akciju dawidd6/action-send-mail
NZ-1	Primjenjivost na nestrukturirane GitHub Actions zapise.	Primjenjivost na nestrukturirane zapise je ostvarena Autoenkoderom koji pretvara sve zapise u vektore iste dužine.
NZ-2	Smanjena ovisnost o ručno definiranim predlošcima zapisa i otpornost na promjene oblika zapisa.	Otpornost na promjene oblika zapisa ostvarena je korištenjem autoenkodera koji ne ovisi o definiranim predlošcima nego pretvara znakove niza izravno u vektor.
NZ-3	Provjerljivost na stvarnim GitHub Actions zapisima.	Provjerljivost na stvarnim podacima ostvarena je označavanjem anomalija u podskupu skupa GHALogs podataka.
NZ-4	Mogućnost uključivanja u CI/CD radni tok uz prihvatljiva i jasno navedena ograničenja izvedbe.	Mogućnost uključivanja u radni tok je ostvarena namjenskom GitHub Akcijom i Python skriptom koja predaje zapise učenim modelima transformera. Razvijena GitHub Akcija je ovisna o instalaciji potrebnih Python paketa.

Osim implementacijskih zahtjeva, za provjeru GHA-AIOps pristupa određeni su i uvjeti eksperimentalne evaluacije. Pristup se najprije ispituje na umjetno generiranom označenom skupu podataka s kontrolirano umetnutim anomalijama, a

zatim se provjerava na označenom podskupu stvarnih GitHub Actions zapisa iz skupa podataka GHALogs.

Zahtjevi usmjereni na oblikovanje i ekstrakciju podataka ostvareni su postupcima opisanim u prethodnom poglavlju, dok su zahtjevi vezani uz arhitekturu mreža, detekciju anomalija i orkestraciju ispunjeni kroz komponente prikazane u nastavku ovog poglavlja.

4.2. Kodiranje zapisa autoenkoderom

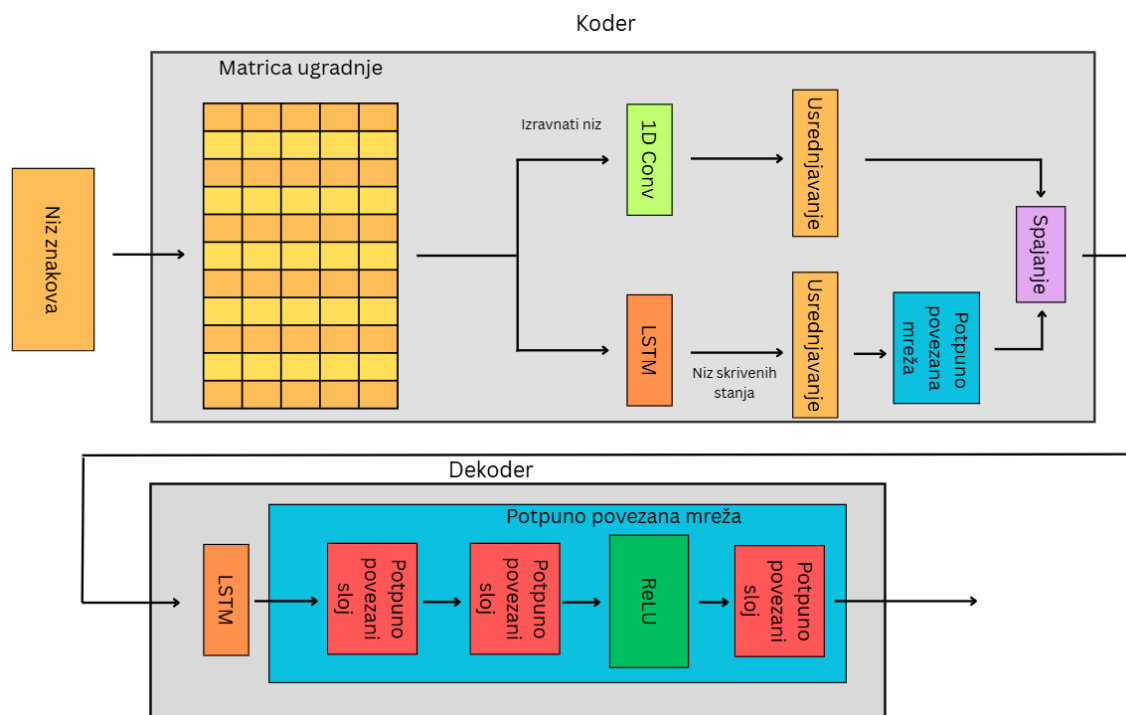
U GHA-AIOps pristupu autoenkoder se koristi za pretvaranje pojedinačnih redaka GitHub Actions zapisa u vektore fiksne duljine. Predobrada zapisa, uklanjanje vremenskih oznaka i pomoćnih oznaka anomalija te početno znakovno kodiranje opisani su u poglavlju 3.3. U ovom se potpoglavlju zato opisuje arhitektura autoenkodera i način na koji se iz predobrađenog zapisa dobiva vektorska reprezentacija prikladna za obradu modelima transformera.

Autoenkoder je odabran zato što omogućuje dobivanje numeričkih reprezentacija nestrukturiranih zapisa bez oslanjanja na unaprijed definirane predloške zapisa. Takav je pristup prikladan za GitHub Actions zapise jer njihov oblik ovisi o korištenim alatima, bibliotekama, skriptama i akcijama unutar pojedinog radnog toka.

Autoenkoder je tip neuronske mreže čiji je cilj kodirati ulaz u smislenu i komprimiranu reprezentaciju te ga zatim rekonstruirati što je bolje moguće [22]. Osnovni dijelovi autoenkodera su koder i dekoder. Koder služi tome da stvara kodiranu reprezentaciju, a dekoder rekonstruira ulaz.

4.2.1. Arhitektura autoenkodera

Arhitektura razvijenog autoenkodera je prikazana na slici 4.1. Ovaj autoenkoder radi na pojedinom zapisu. Svaki zapis je prikazan kao niz brojeva koji se pretvara u niz vektora koristeći matricu ugradnje. Matrica ugradnje se koristi kao tablica koja povezuje broj i vektor. Kako bi se dobila semantička povezanost između znakova koristi se konvolucijski sloj i neuronska mreža s dugotrajnom kratkoročnom memorijom (LSTM).



Slika 4.1 Arhitektura autoenkodera

Prije nego što vektori prođu kroz konvolucijski sloj poredani su jedan za drugim u jedan veliki vektor. Konvolucijski sloj sadrži takvu jezgru da pokriva nekoliko znakova i takav korak da se znakovi ne preklapaju. Kako bi se dobili semantički odnosi među skupinama znakova koristi se usrednjavanje. LSTM je povratna neuronska mreža koja obrađuje podatke poredane u niz. Matematički opis LSTM ćelije se nalazi u dodatku B.

Koder na ulaze LSTM ćelije predaje vektore matrice ugradnje. LSTM ćelija na izlaz daje vektore za koje svaki predstavlja pojedini znak u odnosu na njemu prethodne znakove. Kako bi se dobio prikaz cijelog zapisa koristi se usrednjavanje. Kako bi se vektori velikih dimenzija koji nastaju iz LSTM ćelije smanjili, koriste se dva potpuno povezna sloja, zatim ReLU aktivacija i još jedan potpuno povezani sloj. Konačni izlazi iz LSTM grane i konvolucijske grane se spajaju u jedan vektor koji predstavlja zapis.

Dekoder se sastoji od LSTM ćelije i potpuno povezanih slojeva. Kao ulaz u LSTM ćeliju se koristi izlaz kodera pomnožen s korijenom dužine vektora za lakše razlikovanje sličnih zapisa. Svaki izlaz LSTM ćelije se provodi kroz potpuno povezane slojeve iste arhitekture kao u koderu. Konačni potpuno povezani sloj daje

vektor dimenzija jednak broju mogućih različitih znakova. Primjenom softmax i argmax funkcije moguće je odrediti koji znak se pokušava rekonstruirati.

4.2.2. Učenje autoenkodera i vektorski prikaz zapisa

Za učenje razvijenih modela korišten je postupak gradijentnog spusta. Gradijentni spust je optimizacijski postupak kod kojeg se kreće od neke početne točke w pa se postepeno spušta niz površinu funkcije gubitka u smjeru suprotnome gradijentu u točki w . Kod učenja model w predstavlja vrijednosti težinskih matrica u svakome koraku spusta. Postupak se ponavlja dok vrijednost gradijenta ne iznosi nula ili nije značajno veći od nule.

Učenje autoenkodera se odvija odvojeno od modela koji se koristi za određivanje anomalija. Od učenog modela uzima se samo koder koji se koristiti za stvaranje ulaznih podataka. Kao funkcija gubitka za autoenkoder se koristi unakrsna entropija (engl. cross entropy) prikazana na izrazu (1). Gdje C predstavlja ukupan broj kategorija, kod ovog autoenkodera to je ukupan broj mogućih znakova. Pripadnost ulaznog podataka nekoj kategoriji se određuje binarnom varijablom y_c koja ima vrijednost 1 ako primjer pripada kategoriji c , a inače ima vrijednost 0. Vjerojatnost da primjer pripada oznaci c prikazana je oznakom p_c koja se kod ovog autoenkodera dobije primjenom softmax funkcije na izlaz dekodera. Unakrsna entropija je često korištena funkcija gubitka kad postoje diskretne kategorije podataka.

$$L = - \sum_{c=1}^C y_c \ln(p_c) \quad (1)$$

Učeni autoenkoder pretvara zapise u vektore dimenzije 128, svaki ulazni znak je prikazan vektorom dimenzije 32, konvolucijski sloj grupira zajedno 4 znaka. Koder i Dekoder sadrže dvije uzastopne LSTM ćelije dimenzija 196 kod kodera i 384 kod dekodera.

Dobiveni vektorski prikazi zapisa predstavljaju ulaz u sljedeću fazu GHA-AIOps pristupa. Pojedinačni vektori grupiraju se u klizne prozore, kako je opisano u poglavlju 3.3.3, te se zatim prosljeđuju modelima transformera. Time autoenkoder povezuje nestrukturirane tekstualne zapise i modele koji zahtijevaju numerički oblikovan ulaz.

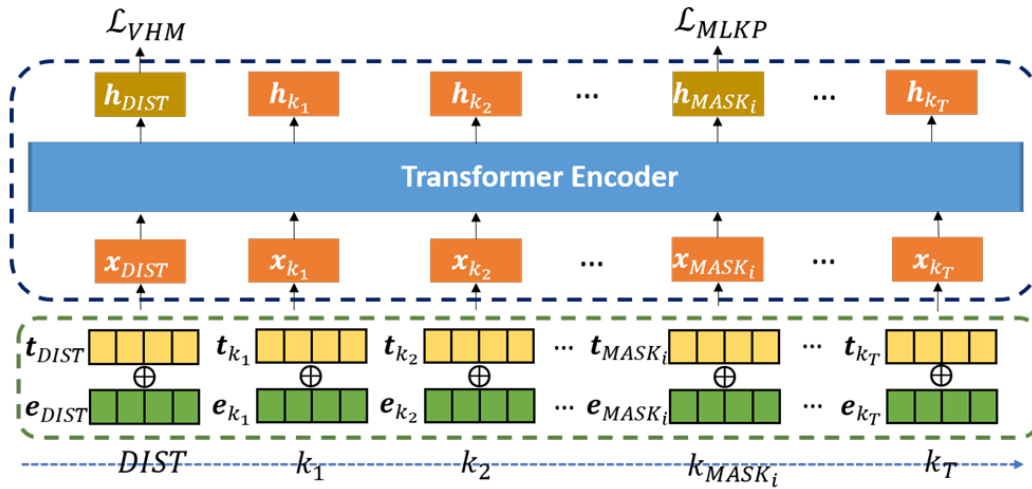
4.3. Modeli transformera za detekciju anomalija

U GHA-AIOps pristupu modeli transformera koriste se za obradu sljedova vektorskih reprezentacija zapisa. Pojedinačni redci zapisa prethodno se kodiraju autoenkoderom, a zatim se grupiraju u klizne prozore opisane u poglavlju 3.3.3. Na taj se način modelima ne predaje samo izolirani zapis, nego slijed zapisa u kojem se može očitovati kontekst izvođenja GitHub Actions posla.

Transformer je prikladan za ovaj zadatak jer mehanizam samopažnje omogućuje modeliranje odnosa među elementima slijeda. U kontekstu zapisa radnog toka to znači da odluka o pojedinom zapisu može ovisiti o zapisima koji mu prethode ili slijede unutar istog prozora [19]. Detaljniji opis opće arhitekture transformera dan je u Dodatku C, dok se u ovom potpoglavlju opisuje njegova primjena u dvama modelima razvijenima u ovom radu.

Za uporabu transformera u detekciji anomalija u zapisima razmatran je model LogBERT [24]. On primjenjuje dvosmjerne koderske reprezentacije iz transformera za učenje uobičajenih obrazaca u zapisima te za otkrivanje odstupanja od tih obrazaca. Njegova je osnovna pretpostavka da se anomalijski zapisi mogu prepoznati kao elementi slijeda koji se ne uklapaju u naučeni kontekst normalnog ponašanja.

U LogBERT pristupu zapisi se najprije parsiraju u predloške zapisa, nakon čega se nad tako dobivenim reprezentacijama provodi učenje modela. U ovom radu zadržava se ideja kontekstualne obrade slijeda zapisa transformerom, ali se napušta oslanjanje na ručno ili algoritamski izvedene predloške zapisa. Umjesto toga, pojedinačni redci zapisa kodiraju se autoenkoderom u vektore fiksne duljine. Time se u GHA-AIOps pristupu smanjuje ovisnost o krutim predlošcima zapisa i omogućuje obradu heterogenih GitHub Actions zapisa. Arhitektura LogBERT-a prikazana je na slici 4.2.



Slika 4.2 Prikaz LogBERT Arhitekture [24]

Zapisi se parsiraju u predloške iz kojih se izvlače značajke. Svakome nizu zapisa se na početak dodaje posebni DIST token. DIST token predstavlja cijeli niz zapisa. Nasumično se odabiru zapisi u nizu kojima se vrijednost maskira. Na vektorske reprezentacije zapisa se dodaje pozicijsko kodiranje. Vektorske reprezentacije se predaju koderu transformera koji računa njihove latentne reprezentacije. Korištenjem potpuno povezanog sloja i softmax funkcije određuje se kojem uzorku maskirani zapis pripada. Latentna reprezentacija DIST se koristi kako bi se latentne reprezentacije zapisa bez anomalija grupirali oko nje, a anomalijski zapisi bi bili udaljeni. Kod ispitivanja zapisa se za svaki maskirani zapis uzima g najvjerojatnijih uzoraka kojima bi zapis mogao pripadati. Ako se u odabranim zapisima nalazi stvarni uzorak tada se zapis smatra normalnim, inače se smatra anomalijom. Ako se u nizu nalazi više od r anomalijskih zapisa tada se cijeli niz smatra anomalijom.

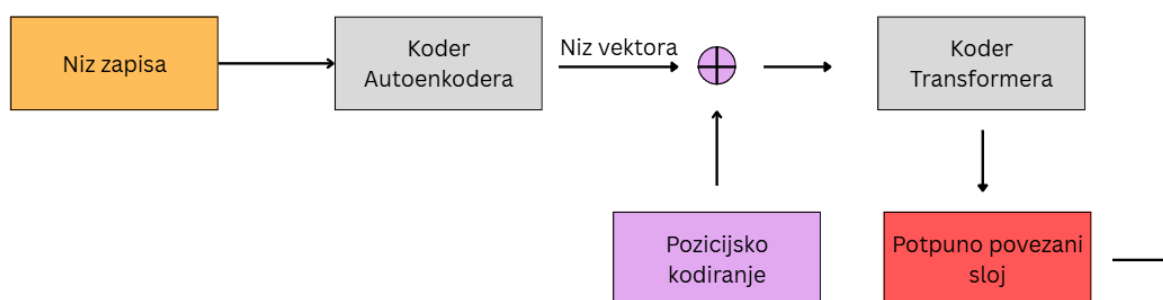
Modeli temeljeni na transformira učinkovito izvlače međuovisnosti podataka u nizu i mogu donositi odluke na razini pojedinog zapisa ili više zapisa. Koder transformera ima sposobnost hvatanja konteksta cijelog niza koji je važan za određivanja anomalija.

Na temelju toga razvijena su dva modela transformera. Prvi model provodi višeklasnu klasifikaciju i svakom zapisu pridružuje jednu od promatranih klasa i primjenjiv je na bilo koji radni tok i koristi koder autoenkodera za čitanje zapisa. Drugi model rekonstruira ulazni slijed vektora i anomalije određuje na temelju rekonstrukcijske pogreške, a za razliku od LogBERT-a ne koristi parsiranje nego

autoenkoder isto kao višeklasni model. Oba modela koriste isti oblik ulaza, ali se razlikuju prema cilju učenja, izlazu i načinu interpretacije rezultata.

4.3.1. Višeklasni model transformera

Višeklasni model temelji se na koderskom dijelu transformera [23]. Dekoder se ne koristi jer cilj modela nije generiranje novog slijeda, nego razvrstavanje postojećih zapisa. Izlazi koda prosljeđuju se potpuno povezanom sloju koji za svaki zapis daje pet vrijednosti, odnosno procjene pripadnosti standardnoj klasi i četirima anomalijским klasama. Arhitektura višeklasnog modela je prikazana na slici 4.3.



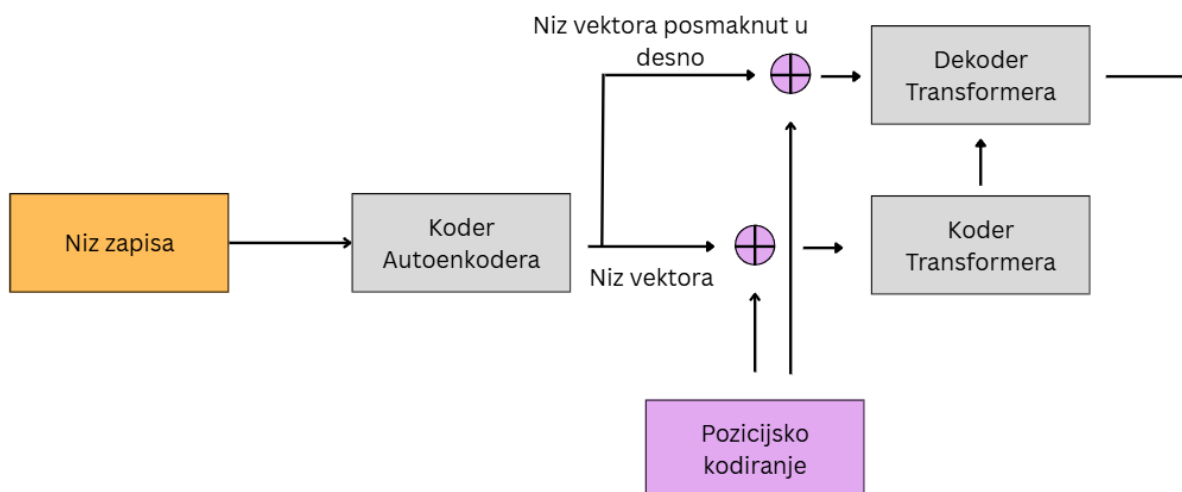
Slika 4.3 Arhitektura višeklasnog modela

Višeklasni model uzima izlaze koda transformera i predaje ih izravno potpuno povezanom sloju. Za svaki zapis iz niza nastaje jedan vektor s pet vrijednosti koje se koriste za izračun procjene pripadnosti navedenim klasama.

4.3.2. Rekonstrukcijski model transformera

Drugi model u GHA-AIOps pristupu jest rekonstrukcijski model transformera. Za razliku od višeklasnog modela, ovaj model ne određuje tip anomalije, nego procjenjuje odstupanje ulaznog slijeda od očekivanog obrasca ponašanja. Time se koristi kao binarni pristup detekciji anomalija, odnosno kao model koji razlikuje standardno i odstupajuće ponašanje.

Rekonstrukcijski model koristi koder i dekoder, ali je konačni potpuno povezani sloj uklonjen. Arhitektura rekonstrukcijskog modela prikazana je na slici 4.4.



Slika 4.4 Arhitektura rekonstrukcijskog modela

Umjesto klasifikacije kao kod prijašnjeg modela očekivani izlaz ovog modela je niz vektora. Cilj ovog modela je da niz izlaznih vektora bude što sličniji ulaznome. Niz zapisa koji ulaze u dekodera je posmaknut u desno i na prvo mjesto je postavljen vektor iste duljine koji se sastoji samo od nula. Model je učen zapisima bez anomalija kako bi naučio njihovu očekivanu raspodjelu.

Određivanje anomalija se radi uspoređivanjem ulaznih i izlaznih vektora. Ako je razlika dovoljno velika zapis se smatra anomalijom. Granična vrijednost rekonstrukcijske pogreške korištena kod integracije modela u radni tok određuje se postupkom opisanom u poglavlju 4.3.3. Rekonstrukcijski model ovisi o radnom toku na kojem je učen i potrebno ga je periodički dodatno podučiti.

Ograničenje rekonstrukcijskog modela jest njegova ovisnost o distribuciji podataka skupa na kojem je učen. Promjena sadržaja radnog toka na kojem je model učen će povećati pogrešku rekonstrukcije. Kako bi smanjilo pojavljivanje lažno pozitivnih rezultata model je potrebno periodički dodatno učiti na zapisima koji ne sadrže anomalije da se izbjegne lažni alarm.

4.3.3. Učenje modela transformera

Za učenje oba transformer modela korišten je postupak gradijentnog spusta. Višeklasni model koristi kao funkciju gubitak unakrsnu entropiju prikazanu izrazom (1). Različito od autoenkodera ovdje se računa vjerojatnost da zapis pripada nekoj od kategorija anomalija ili je standardan. Učeni model na ulaz prima vektore dimenzija 128, sastoji se od dvoslojnog kodera s dvije glave samopažnje (engl. self-

attention heads). Za vrijeme učenja korišten je prozor veličine 30 i pomak veličine 10.

Za razliku od prijašnjih modela rekonstrukcijski model transformera ne koristi funkciju unakrsne entropije. Kao funkciju gubitka koristi srednji kvadratni gubitak prikazan izrazom (2).

$$L = \frac{1}{n} \sum_{n=1}^N (y_n - \hat{y}_n)^2 \quad (2)$$

Gdje N predstavlja dimenziju vektora, y_n predstavlja stvarnu vrijednost vektora na mjestu n , a $\hat{y}_n$ predstavlja predviđenu vrijednost. Ova funkcija gubitka mjeri međusobnu udaljenost vektora u visoko dimenzionalnom prostoru. I ovaj model učen je gradijentnim spustom. Učeni model na ulaz prima vektore dimenzija 128, sastoji se od dvoslojnog koder i dekoder s dvije glave samopažnje. Za vrijeme učenja korišten je prozor veličine 30 i pomak veličine 10.

Za određivanje rekonstrukcijskog praga korištenog u GitHub akciji koja integrira ovaj model u radni tok koristiti se skup podatak za učenje. Na skupu podatak za učenje računa se prosječna vrijednost i standardna devijacija rekonstrukcijska pogreška. Tada se prag određuje prema izrazu $T_{op} = \mu_{train} + 2\sigma_{train}$. Gdje μ predstavlja srednju vrijednost rekonstrukcijske pogreške, a σ standardnu devijaciju na skupu za učenje. Ovaj prag je izabran heuristički kako ne bi bio preosjetljiv na primjene u radnom toku, ali bi i dalje mogao odrediti anomalije. Ovim postupkom je dobivena srednja vrijednost 0,037119, standardna devijacija 0,1757 i prag 0,38852. Obzirom da prag ovisi o raspodjeli zapisa radnog toka, nakon značajne promjene radnog toka ili strukture zapisa potrebno ga je ponovno odrediti.

Tablica 4.2 sadrži sažeti prikaz parametara modela i njihove funkcije gubitka.

Tablica 4.2 Prikaz parametara učenih modela

Model	Parametri modela	Funkcija gubitka
autoenkoder	matrica ugradnje s vektorima duljine 32, konvolucija grupira 4 znaka, koder i dekoder s dvije LSTM ćelije dimenzija 196 za koder i 384 za dekoder, zapis prikazan vektorom duljine 128.	unakrsna entropija
višeklasni model	ulazni vektor duljine 128, dvoslojni koder dimenzije 1024 s dvije glave samopažnje, prozor veličine 30 pomak 10	unakrsna entropija

rekonstrukcijski model	ulazni vektor duljine 128, dvoslojni koder i dekode dimenzije 1024 s dvije glave samopažnje, prozor veličine 30 pomak 10	srednji kvadratni gubitak
---------------------------	--	------------------------------

4.3.4. Uloga dvaju modela u GHA-AIOps pristupu

Višeklasni i rekonstrukcijski model imaju komplementarne uloge u GHA-AIOps pristupu. Višeklasni model koristi se kada je cilj ne samo detektirati anomaliju, nego i odrediti njezin tip. Takav je model prikladan u slučajevima kada postoje označeni podaci za promatrane klase anomalija i kada je korisniku važno dobiti interpretabilniju oznaku nalaza.

Rekonstrukcijski model ima drukčiju ulogu. On se temelji na učenju standardnog ponašanja i ne zahtijeva oznake za sve tipove anomalija. Zbog toga je prikladan za detekciju odstupanja koja nisu nužno unaprijed razvrstana u poznate klase. Njegovo je ograničenje to što rezultat nije tip anomalije, nego vrijednost rekonstrukcijske pogreške i odluka o odstupanju. Ograničenje takvog pristupa jest njegova ovisnost o raspodjeli podataka na kojima je model učen, što se dodatno razmatra u poglavljima 5.3 i 5.4.

Kombinacijom ovih dvaju modela GHA-AIOps pristup objedinjuje klasifikacijski i rekonstrukcijski pogled na zapise GitHub Actions radnih tokova. Višeklasni model omogućuje razlikovanje unaprijed definiranih anomalijских kategorija, dok rekonstrukcijski model omogućuje procjenu odstupanja od naučenog standardnog obrasca. Time se povećava fleksibilnost pristupa u odnosu na uporabu samo jednog modela.

4.4. Integracija u GitHub Actions i ograničenja izvedbe

Radni tokovi automatiziraju izgradnju i isporuku programske podrške. Kako bi izrađeni modeli bili učinkoviti i njih je potrebno uključiti u radni tok. Svaki korisnik GitHuba može izraditi javno dostupnu Akciju. U praksi se koristi mali broj takvih Akcija kako bi se izbjegli sigurnosni rizici. Za potrebe integracije modela opisanih u ovome radu izrađena je javno dostupna Akcija. Javno dostupni repozitorij sadrži naučene modele i kod potreban za njihovo pokretanje i analizu zapisa.

4.4.1. Organizacija programskih komponenti

Programsko rješenje organizirano je kao skup komponenti koje zajedno ostvaruju tok detekcije anomalija u GitHub Actions okruženju. Svaka komponenta ima jasno određenu ulogu u obradi. Actions.yml predstavlja konfiguracijsku datoteku koja definira strukturu i korake izvođenja same GitHub akcije. Ona služi kao ulazna točka i upravlja redoslijedom izvršavanja svih ostalih komponenti. Za dohvaćanje zapisa iz radnog toka koristi se skripta fetch_logs.py. Primanjem autorizacijskog tokena i naziva posla, skripta koristi GitHub REST API kako bi preuzela relevantne zapise izvršenog posla. Važan je i korak instalacije Python paketa koji omogućuju ispravan rad i pokretanje modela za detekciju anomalija. Detekcija anomalija se pokreće skriptom detect.py. Osim pregleda zapisa ova komponenta generira konačan izvještaj o pronađenim nepravilnostima. Na kraju radnog toka se koristi standardna GitHub Akcija dawidd6/action-send-mail za automatizirano slanje e-pošte s priloženim izvještajem prema definiranim primateljima. Sažeti opis glavnih programskih komponenti prikazan je u tablici 4.3.

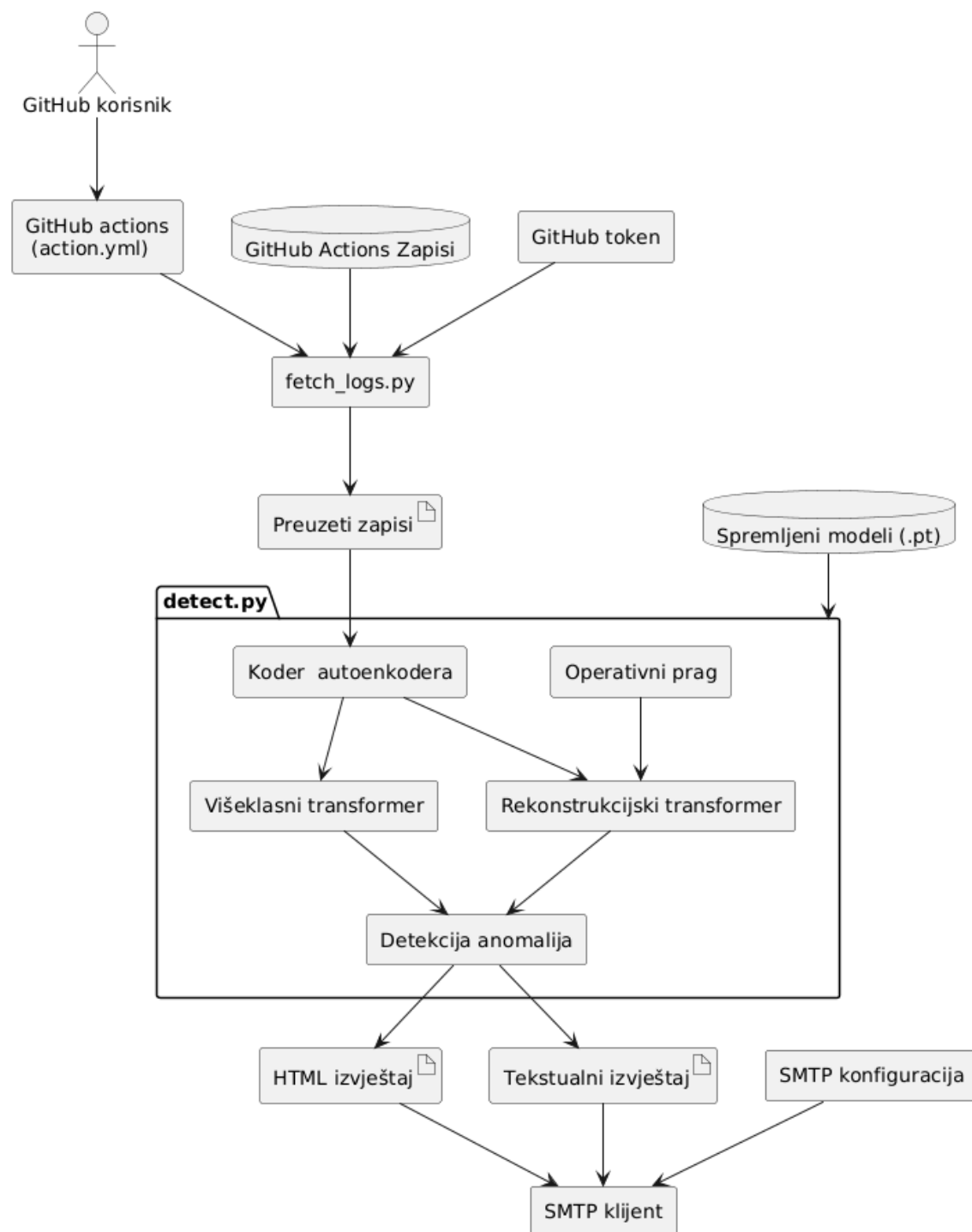
Tablica 4.3 Glavne programske komponente GHA-AIOps pristupa

Komponenta	Uloga	Ulaz	Izlaz
actions.yml	Definira korake GitHub Akcije i redoslijed njihova izvođenja.	Parametri akcije, naziv posla, pristupni podaci	Pokretanje dohvata zapisa, detekcije i slanja obavijesti
fetch_logs.py	Pristupa GitHub REST API-u za dohvaćanje zapisa koje je potrebno obraditi.	Autorizacijski token s dozvolom čitanja i naziv posla koji se obrađuje.	Datoteka koja sadrži zapise zadanog posla
detect.py	Obrada zapisa i izrada izvještaja	Put do mjesta preuzimanja akcije i naziv relevantnog posla	Html datoteka s izvještajem i txt datoteka koja sadrži anomalije
dawidd6/action-send-mail	Služi za automatizirano slanje e-pošte s izvještajem	Pristupni podaci SMTP servera za slanje pošte i adresa primatelja	E-pošta koja sadrži izvještaj

Izlazni artefakt: Izveštaj	Služi za obavještanje korisnika ili razvojnog tima o pronađenim anomalijama	Zapisi analizirani korištenjem autoenkodera i transformera	Tipovi anomalija pronađeni u radnom toku s redcima od kojih se sastoje anomalije.
-------------------------------	---	--	---

Uz tablični prikaz glavnih programskih komponenti, na slici 4.5 prikazan je komponentni tok integracije GHA-AIOps pristupa u GitHub Actions radni tok. Dijagram prikazuje kako korisnik uključuje razvijenu GitHub Akciju u konfiguracijsku datoteku `actions.yml`, nakon čega se s pomoću skripte `fetch_logs.py` dohvaćaju zapisi odabranog posla. Preuzeti zapisi zatim se predobrađuju i predaju modelskom modulu, u kojem se učitavaju spremljeni modeli te se provodi kodiranje zapisa i detekcija anomalija.

Modelske modul obuhvaća koder autoenkodera, višeklasni model transformera i rekonstrukcijski model transformera. Koder autoenkodera pretvara retke zapisa u vektorske reprezentacije, dok modeli transformera nad tako oblikovanim sljedovima provode klasifikacijsku ili rekonstrukcijsku detekciju anomalija. Rezultati detekcije zatim se obrađuju skriptom `detect.py`, iz koje nastaju HTML izvještaj i tekstualni izvještaj. HTML izvještaj koristi se kao sadržaj obavijesti e-poštom, dok tekstualni izvještaj sadrži detaljniji prikaz označenih dijelova zapisa.



Slika 4.5 Komponentni prikaz integracije GHA-AIOps pristupa u GitHub Actions radni tok

Ovakva podjela jasno razdvaja odgovornosti pojedinih komponenti. Skripta `fetch_logs.py` zadužena je za dohvat ulaznih podataka, modeli za numeričku obradu i detekciju anomalija, a skripta `detect.py` za povezivanje obrade zapisa s izlaznim izvještajima. GitHub Akcija pritom služi kao integracijski sloj koji omogućuje izvođenje cijelog postupka unutar CI/CD radnog toka.

Ova akcija odvija se u vlastitom poslu čije je pokretanje uvjetovano poslovima čiji se zapisi analiziraju. Isječak actions.yml datoteke koji se odnosi na korake akcije je prikazan u kôdu 4.1.

```
runs:
  using: "composite"
  steps:
    - name: Fetch logs
      shell: bash
      run: python ${github.action_path}/fetch_logs.py -
t ${inputs.github-token}} -j ${inputs.job-name}}
    - name: Install dependencies
      shell: bash
      run: pip install torch
    - name: Detection
      shell: bash
      id: detector
      run: |
        python ${github.action_path}/detect.py -j
${inputs.job-name}} -p ${github.action_path} > anomaly.txt
        if [ -f body.html ]; then
          echo "detection=true" >> $GITHUB_OUTPUT
        else
          echo "detection=false" >> $GITHUB_OUTPUT
        fi
```

Kôd 4.1 Koraci akcije koja detektira anomalije

4.4.2. Dohvat zapisa i pokretanje detekcije

Proces započinje pokretanjem Python skripte `fetch_logs.py`, kojoj se kao parametri prosleđuju GitHub token za autorizaciju pristupa i naziv samog posla. Skripta najprije koristi GitHub REST API za dohvaćanje popisa svih poslova unutar radnog toka u kojemu se akcija izvodi. Iz dobivenog se popisa potom izdvaja jedinstveni identifikator traženog posla. U slučaju da se identifikator ne pronađe, izvršavanje akcije se prekida uz generiranje pogreške. Ako je identifikator uspješno pronađen, s pomoću njega se pristupa zapisima zadanog posla, koji se nakon preuzimanja pohranjuju u `.txt` datoteku na poslužitelju.

Nakon uspješnog dohвата i pohrane zapisa, pokreće se preuzimanje Python paketa `torch`, potrebnog za ispravno pokretanje modela. S obzirom na to da preuzimanje ovog paketa može biti vremenski zahtjevno, ovaj je korak namjerno postavljen nakon faze dohvaćanja zapisa. Time se smanjuje potrošnja resursa, jer u slučaju pogreške kod dohвата zapisa paket `torch` nije potrebno instalirati.

Korak detekcije anomalija pokreće skriptu `detect.py`. Kao ulazni parametri, skripti se proslijeđuju naziv posla i put do repozitorija akcije unutar virtualnog okruženja. Navedeni put omogućuje skripti lokalno učitavanje prethodno istreniranih modela autoenkodera i transformera. Po završetku učitavanja modela, pohranjeni se zapisi čitaju te se strukturiraju u klizne prozore. Ti se prozori slijedno proslijeđuju modelima koji provode analizu i detekciju anomalija.

4.4.3. Izrada izvještaja i slanje obavijesti

Izrada izvještaja se odvija istovremeno s analizom i detekcijom anomalija. Kada se u pojedinom kliznom prozoru detektiraju anomalije ispisuje se redni broj linije kojom taj prozor započinje, nakon čega se ispisuje cjelokupan sadržaj prozora s jasno označenim anomalijama. Prednost kod označavanja anomalija ima višeklasni model jer on može točno navesti tip anomalija. Kao odluku višeklasnog modela se uzima onaj tip anomalije kojem je dodijeljena najveća vrijednost. Ako višeklasni model nije detektirao anomaliju tada se provjerava rekonstrukcijska pogreška rekonstrukcijskog modela. Ako je pogreška veća od 0,38852 tada se taj zapis označava kao nepoznata anomalija. Postupak određivanja praga opisan je u poglavlju 4.3.3. Standardni izlaz (engl. `standard output`) skripte preusmjerava se i pohranjuje u `.txt` datoteku. Nakon završetka cjelokupne analize, generira se izvještaj u HTML formatu (`body.html`) koji sadrži pregled uočenih tipova anomalija. Navedena HTML datoteka stvara se i pohranjuje u slučaju da su tijekom analize detektirane anomalije. Prikaz aktivnosti detekcije i izrade izvještaja se nalazi na slici 4.6.



Slika 4.6 Dijagram aktivnosti detekcije, izrade izvještaja i slanja obavijesti

Korisnici akcije su obaviješteni o ishodu detekcije putem e-pošte. Kôd 4.2 prikazuje isječak actions.yml datoteke koji je odgovoran za slanje e-pošte.

```

- name: Send mail notification
  uses: dawidd6/action-send-mail@v3
  if: steps.detector.outputs.detection == 'true'
  with:
    server_address: ${inputs.server-address}}
    server_port: 465
    secure: true
    username: ${inputs.mail-username }}
    password: ${inputs.mail-password }}
    to: ${inputs.mail-address}}
    subject: "Anomaly detection alert"
    from: GitHub Log Bot <${inputs.mail-username }>
    html_body: file://body.html
    attachments: anomaly.txt

```

Kôd 4.2 Korak akcije slanja e-pošte

Izvršavanje ovog koraka izravno je uvjetovano postojanjem datoteke body.html. Za implementaciju slanja elektroničke pošte koristi se javno dostupna GitHub Akcija dawidd6/action-send-mail. Sadržaj same poruke definiran je generiranom HTML datotekom, dok se .txt datoteka s detaljno označenim anomalijama prilaže kao privitak (engl. attachment).

Kôd 4.3 prikazuje primjer integracije razvijene akcije u radni tok. Kako bi se razvijena akcija uspješno integrirala, potrebno ju je konfigurirati unutar zasebnog posla koji eksplicitno ovisi o poslu čiji se zapisi analiziraju (s pomoću ključne riječi needs). Kao ulazni parametri akciji potrebno je predati ime posla i token s dozvolom čitanja. Za obavještavanje putem e-pošte potrebno je predati adresu primatelja obavijesti. Predaju se korisničko ime i zaporka kojima se može pristupiti SMTP (engl. Simple Mail Transfer Protocol) poslužitelju. SMTP poslužitelj koristi se kao infrastruktura za slanje e-pošte, pri čemu se u zaglavlju elektroničke pošte kao pošiljatelj prikazuje naziv "GitHub Log Bot", uz pripadajuće korisničko ime navedeno u zagradi.

```

analyze-logs:
  runs-on: ubuntu-latest
  needs: build-and-test
  permissions:
    actions: read
  steps:
    - name: Run Anomaly Detector Action
      uses: DominikOzvald/do-dr-models@main
      with:
        github-token: ${secrets.GITHUB_TOKEN }}
        job-name: "build-and-test"
        mail-address: "dominik.ozvald@gmail.com"
        mail-username: ${secrets.MAIL_USERNAME}}
        mail-password: ${secrets.MAIL_PASSWORD}}
        server-address: smtp.gmail.com

```

Kôd 4.3 Primjer integracije akcije u radni tok

4.4.4. Ograničenja izvedbe

Iako implementirana GitHub Akcija pokazuje da se GHA-AIOps pristup može uključiti u stvarni radni tok, postoje i određena ograničenja izvedbe. Prvo ograničenje odnosi se na vrijeme izvođenja. Ovisnost akcije o Python paketu torch povećava vrijeme izvođenja za približno dvije minute. Moguće rješenje ovog ograničenja je izrada unaprijed prevedenog koda ili kontejnera koji bi već sadržavao cjelokupno izvršno okruženje i kod potreban za analizu i detekciju anomalija, čime bi se izbjeglo ponovno instaliranje ovisnosti.

Implementirana GitHub Akcija pretpostavlja da su zapisi analiziranog posla dostupni nakon završetka izvođenja te da je naziv posla ispravno zadan. Ako se naziv posla ne podudara s nazivom posla u radnom toku, dohvat zapisa nije moguće provesti. Osim toga, predani GitHub token mora imati ovlasti potrebne za čitanje zapisa radnog toka. U suprotnome se dohvat zapisa prekida prije pokretanja modela za detekciju anomalija.

Dodatno ograničenje je da se detekcije i analiza izvode nakon završetka posla čiji se zapisi obrađuju. Kod nekih tipova anomalija na primjer tihih neuspjeha bi bilo poželjno zaustaviti radni tok čim se anomalija pojavi u zapisima. Ovakav pristup ne podržava obradu zapisa u stvarnom vremenu tijekom njihovog nastajanja. Treće ograničenje odnosi se na opseg detekcije anomalija. Generirani izvještaj ograničen je na tipove anomalija prikazane u ovome radu. Operativni prag ovisi o raspodjeli zapisa radnog toka na kojem je određen. Promjena radnog toka ili strukture zapisa može povećati rekonstrukcijsku pogrešku i broj lažnih uzbuna, zbog čega prag može zahtijevati ponovno određivanje.

Rezultati detekcije trebaju se tumačiti kao pomoć razvojnome timu u usmjeravanju pozornosti na potencijalno rizične dijelove zapisa, a ne kao potpuna automatizirana analiza uzroka problema. Zbog toga se GHA-AIOps pristup promatra kao potpora inženjerskoj analizi, pri čemu konačno tumačenje nalaza, procjenu njihove važnosti i odluku o potrebnim korektivnim postupcima i dalje treba provesti razvojni tim. Unatoč navedenim ograničenjima, integracija pokazuje da se GHA-AIOps pristup može povezati s postojećim GitHub Actions radnim tokovima i koristiti za automatsko označavanje potencijalno rizičnih obrazaca u zapisima. U sljedećem

poglavlju vrednuje se uspješnost razvijenih modela na umjetno generiranom ispitnom skupu i označenom podskupu stvarnih GHALogs zapisa.

5. Eksperimentalna evaluacija GHA-AIOps pristupa

U ovom su poglavlju prikazana je eksperimentalna evaluacija predloženog GHA-AIOps pristupa za detekciju anomalija u zapisima GitHub Actions radnih tokova. Opisane su korištene mjere uspješnosti te su prikazani rezultati ispitivanja učenih modela. Višeklasni model evaluiran je na umjetno generiranom označenom skupu podataka, kao i na označenom podskupu stvarnih zapisa iz skupa GHALogs. Kako bi se odredila sposobnost modela u otkrivanju rijetkih anomalija s obzirom na veliki broj standardnih zapisa preciznost, odziv i F1-mjera su izračunate zasebno za svaki tip anomalije. Rekonstrukcijski model je ispitan samo na označenim podacima umjetno generiranog skupa jer je vezan uz radni tok na kojem je treniran. Za evaluaciju rekonstrukcijskog modela prikazana je ROC krivulja te je izračunata pripadajuća površina ispod krivulje jer se odluka o anomaliji donosi promjenom praga rekonstrukcijske pogreške.

5.1. Metodologija evaluacije

Kako bi se formalno evaluirala uspješnost predloženog pristupa korištene su mjere uspješnosti temeljene na sposobnosti razlikovanja standardnih i anomalijских zapisa. Temelj za izračun mjera uspješnosti je matrica zabune (engl. confusion matrix).

Evaluacija je provedena na dvama skupovima podataka. Umjetno generirani skup podataka korišten je za ispitivanje modela u kontroliranim uvjetima, s unaprijed poznatim i automatski dodijeljenim oznakama anomalija. Označeni podskup GHALogs zapisa korišten je za provjeru primjenjivosti višeklasnog modela na stvarnim zapisima GitHub Actions radnih tokova. Budući da je rekonstrukcijski model ovisan o raspodjeli zapisa radnog toka na kojem je učen, njegova je evaluacija provedena samo na umjetno generiranom skupu podataka.

S obzirom na izraženu neravnotežu klasa, evaluacija se ne temelji samo na ukupnoj točnosti modela. U GitHub Actions zapisima standardni zapisi znatno su brojniji od anomalijских zapisa, pa visoka ukupna točnost može prikriti slabiju detekciju rijetkih anomalija. Zbog toga su rezultati višeklasnog modela prikazani po pojedinim klasama, uz makro prosjek i matricu zabune. Za rekonstrukcijski model koristi se

ROC krivulja i površina ispod ROC krivulje jer se odluka o anomaliji donosi promjenom praga rekonstrukcijske pogreške.

5.1.1. Eksperimentalni protokol

Eksperimentalni protokol određen je različitim ulogama dvaju modela. Višeklasni model ispitivan je na umjetno generiranom ispitnom skupu i na označenom podskupu stvarnih GHALogs zapisa. Umjetno generirani skup omogućuje provjeru ponašanja modela u uvjetima u kojima su tipovi anomalija kontrolirano ubačeni u zapise. Time se dobiva početna procjena sposobnosti modela da razlikuje standardne zapise i četiri promatrane anomalijske klase. Podskup GHALogs koristi se za provjeru ponašanja modela na heterogenijim stvarnim zapisima, čiji se oblik, rječnik i struktura razlikuju od umjetno generiranih zapisa.

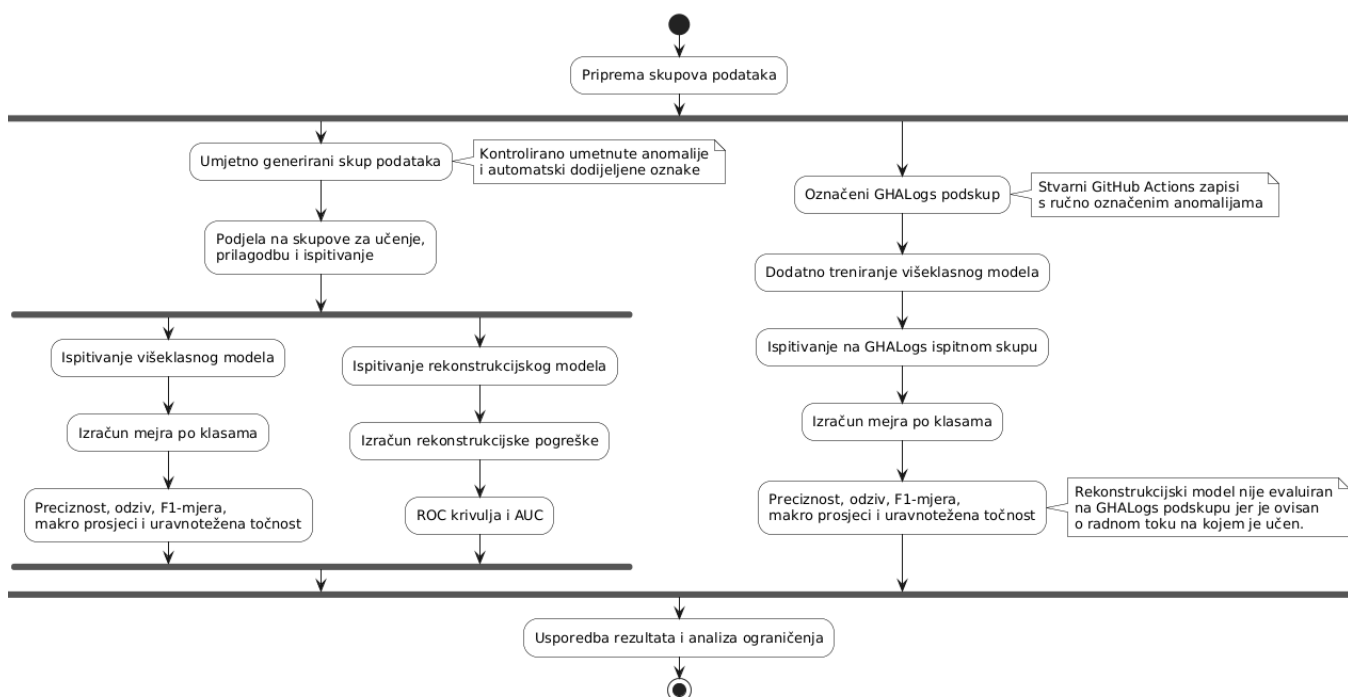
Prilikom evaluacije višeklasnog modela bitno je osigurati jedinstvenu odluku za svaki pojedini zapis. Korištenje preklapajućih prozora kao kod učenja modela značilo bi višestruki prolaz istog zapisa kroz model. To bi potencijalno dovelo do nekonzistentnih predikcija za isti zapis. Kako bi se izbjeglo preklapanje i osigurala jednoznačnost rezultata, u kod ispitivanja modela primijenjen je prozor kod kojeg su veličina prozora i koraka jednake. Konačna odluka modela o klasifikaciji zapisa donosi se primjenom funkcije argmax , odnosno odabirom one klase kojoj je model dodijelio najveću vjerojatnost.

Kako bi se model uspješno prilagodio specifičnostima stvarnih podataka, primijenjen je postupak dodatnog treniranja kroz manji broj epoha. S ciljem regularizacije procesa učenja i sprječavanja pretjeranog udaljavanja od parametara inicijalno učenog modela u podskup za dodatno učenje dodano je i jedno pokretanje simuliranih podataka.

Rekonstrukcijski model evaluiran je drukčijim protokolom jer ne dodjeljuje zapisima jednu od unaprijed definiranih klasa. Model pokušava rekonstruirati ulazni slijed vektora, a anomalija se određuje na temelju razlike između ulaza i rekonstruiranog izlaza. Ako je rekonstrukcijska pogreška veća od odabranog praga, zapis smatra se anomalnim. Budući da različiti pragovi daju različit odnos između otkrivenih anomalija i lažnih uzbuna, uspješnost rekonstrukcijskog modela prikazana je ROC krivuljom. ROC krivulja se određuje pomicanjem praga od takve vrijednosti da su

svi primjeri veći označeni kao standardni do takve da su svi primjeri označeni kao anomalije. Prag se postepeno miče tako da svaki put je još jedan primjer označen kao anomalija. Za ispitivanje rekonstrukcijskog model nije određen jedan prag za koji se računaju preciznost, odziv i F1-mjera nego je njegova uspješnost određena površinom ispod ROC krivulje (engl. Area under curve - AUC).

Redoslijed koraka cijelog eksperimentalnog protokola je prikazana na slici 5.1.



Slika 5.1 Eksperimentalni protokol evaluacije GHA-AIOps pristupa

Nakon pripreme podataka dijagram se razdvaja na dvije glavne grane: lijevu, koja prikazuje korake ispitivanja modela na umjetno generiranim podacima, i desnu, koja prikazuje korake evaluacije na stvarnom skupu podataka. U lijevoj se grani, nakon podjele sintetičkog skupa, tijek dalje dijeli na ispitivanje višeklasnog modela (izračun preciznosti, odziva, F1-mjere i uravnotežene točnosti) te rekonstrukcijskog modela (izračun rekonstrukcijske pogreške i analiza ROC krivulje). Desna grana vizualizira postupak ispitivanja višeklasnog modela na stvarnim podacima, koji uz evaluaciju po klasama uključuje i dodatni korak treniranja modela.

5.1.2. Mjere evaluacije

Temelj za izračun mjera uspješnosti jest matrica zabune. Matrica zabune prikazuje odnos između stvarnih oznaka i oznaka koje je predvidio model. Svaki redak

odgovara jednoj stvarnoj klasi, a stupci predstavljaju klase koje je predvidio model. Vrijednosti na glavnoj dijagonali matrice predstavljaju ispravno klasificirane primjere, dok vrijednosti izvan glavne dijagonale predstavljaju pogrešne klasifikacije.

Kod modela s binarnom klasifikacijom za prikaz rezultata koristi se matrica zabune. Općenita matrica zabune prikazana je izrazom (3).

$$\begin{pmatrix} TP & FN \\ FP & TN \end{pmatrix} \quad (3)$$

Kod binarne klasifikacije razlikuju se četiri osnovna ishoda: stvarno pozitivni primjeri, lažno pozitivni primjeri, stvarno negativni primjeri i lažno negativni primjeri. Oznaka TP predstavlja stvarno pozitivne podatke, FP predstavlja lažno pozitivne podatke, TN stvarno negativne i FN lažno negativne. Ako klasifikacija nije binarna onda se matica proširuje na broj klasa, ali se gube pozitivne i negativne oznake. U području strojnog učenja i dubokog učenja se često kao mjere uspjeha koristi točnost, odziv, preciznost i F1-mjera. Točnost predstavlja udio točno označenih primjera. Točan iznos se računa kao broj točno označenih primjera podijeljen ukupnim brojem primjera. Odziv predstavlja omjer stvarno pozitivno označenih primjera u skupu svih pozitivnih primjera računa se po izrazu (4). Odziv se još nekad naziva i stopa stvarnih pozitiva (TPR) ili osjetljivost.

$$R = TPR = \frac{TP}{TP+FN} \quad (4)$$

Preciznost je omjer stvarno pozitivnih primjera i svih primjera koje je model označio kao pozitivne. Računa se po izrazu (5).

$$P = \frac{TP}{TP+FP} \quad (5)$$

Mjera koja objedinjava mjere preciznosti i odziva naziva se F1-mjera i računa po izrazu (6).

$$F = \frac{2PR}{P+R} \quad (6)$$

Kod višeklasne klasifikacije ove se mjere ne mogu izravno izračunati za sve kase zajedno, već se računaju za svaku klasu zasebno gdje se promatrana klasa smatra pozitivnom, a ostale negativnim. Na taj se način za svaku klasu zasebno mogu izračunati preciznost, odziv i F1-mjera.

Kako bi se utvrdila ukupna uspješnost modela za pojedinu mjeru koristi se makro prosjek koji se računa kao aritmetička sredina vrijednosti dobivenih za svaku klasu. Takav pristup svakoj klasi daje jednaku težinu, neovisno o tome koliko je primjera te klase zastupljeno u skupu podataka. To je važno jer su anomalijске klase znatno rjeđe od standardnih zapisa. U radu se dodatno prikazuje točnost nad anomalijским zapisima. Ta se mjera računa samo nad zapisima čija stvarna oznaka pripada jednoj od anomalijških klasa. Vrijednost predstavlja udio anomalijških zapisa kojima je model ispravno dodijelio njihovu stvarnu anomalijšku klasu. Mjera ne uključuje standardnu klasu, pa izravnije prikazuje uspješnost razvrstavanja anomalija od ukupne točnosti, koja je snažno pod utjecajem velikog broja standardnih zapisa.

U višeklasnom slučaju dodatno se koristi uravnotežena točnost (engl. balanced accuracy), koja odgovara makro prosjeku odziva po klasama. Za razliku od ukupne točnosti, ova mjera svakoj klasi daje jednaku težinu, neovisno o broju njezinih primjera. Za standardnu višeklasnu klasifikaciju uravnotežena točnost računa se kao aritmetička sredina odziva po svim klasama uključenima u evaluaciju:

$$BA = \frac{1}{K} \sum_i^K \frac{TP_i}{TP_i + FN_i} \quad (7)$$

pri čemu je K broj klasa, TP_i broj ispravno prepoznatih primjera klase i , a FN_i broj primjera klase i koje je model pogrešno svrstao u neku drugu klasu. Budući da izraz $\frac{TP_i}{TP_i + FN_i}$ predstavlja odziv za klasu i , uravnotežena točnost u ovom slučaju odgovara makro prosjeku odziva po klasama.

Uravnotežena točnost prikladna je za neuravnotežene skupove podataka jer svakoj klasi daje jednaku težinu i time smanjuje mogućnost da visoka uspješnost na većinskoj standardnoj klasi prikrije slabiju detekciju rjeđih anomalijških klasa. U ovom se radu uravnotežena točnost tumači kao dopuna ukupnoj točnosti, dok se preciznost, F1-mjera i matrica zabune koriste za detaljniju analizu pogrešaka po klasama.

Za modele s pragom se kao mjera uspješnosti koristi površina pod krivuljom ROC. ROC krivulja se dobije tako da se prag postepeno mijenja od veće vrijednosti, takve da su svi primjeri negativni, do manje vrijednosti, takve da su svi primjeri pozitivni. Prag se postepeno miče tako da svaki put je još jedan primjer označen kao

pozitivan. Odziv se nalazi na y osi ROC krivulje, a na x osi se nalazi stopa lažnog alarma koja se računa prema izrazu (8).

$$FPR = \frac{FP}{TN+FP} \quad (8)$$

Za rekonstrukcijski model koristi se ROC krivulja. Rekonstrukcijski model ne daje izravnu klasnu oznaku, nego vrijednost rekonstrukcijske pogreške. Odluka o tome je li zapis anomaliji ovisi o odabranom pragu. Ako je prag nizak, model će detektirati veći broj anomalija, ali će se povećati i broj lažno pozitivnih rezultata. Ako je prag visok, smanjuje se broj lažnih uzbuna, ali se povećava rizik propuštanja stvarnih anomalija. ROC krivulja prikazuje odnos između stope lažno pozitivnih rezultata i stope stvarno pozitivnih rezultata pri različitim vrijednostima praga.

5.2. Evaluacija višeklasnog modela

U ovom potpoglavlju prikazani su rezultati evaluacije višeklasnog modela transformera. Model se vrednuje na umjetno generiranom ispitnom skupu i na označenom podskupu GHALogs, kako bi se usporedilo njegovo ponašanje u kontroliranim i stvarnim uvjetima. Rezultati se najprije prikazuju zasebno za svaki skup podataka, a zatim se analiziraju najvažnije pogreške modela i razlike između umjetno generiranih i stvarnih zapisa.

5.2.1. Rezultati na umjetno generirani skup podataka

Višeklasni model je najprije ispitan na zapisima iz umjetno generiranog skupa podataka koji su podijeljeni u skup za ispitivanje. Na slici 5.2 prikazana je matrica zabune za višeklasni model transformera na umjetno generiranom ispitnom skupu. Matrica je normalizirana po redcima: svaki redak odgovara jednoj stvarnoj klasi, a stupci predstavljaju klase koje je predvidio model. Vrijednost na poziciji i, j predstavlja udio zapisa stvarne klase i koje je model označio kao klasu j . Matrica je normalizirana kako bi se uspješnost modela mogla usporediti po klasama unatoč tome što je broj standardnih zapisa znatno veći od broja anomaljskih zapisa.



Slika 5.2 Matrica zabune na umjetnom ispitnom skupu podataka

U prikazanoj matrici zabune oznaka standard odnosi se na zapise koji ne sadrže anomalije, oznaka nestabilni na nestabilne ispite, oznaka tihi na tihe neuspjehe, a oznaka sigurnost na sigurnosne anomalije. Vrijednosti na glavnoj dijagonali matrice predstavljaju udio ispravno klasificiranih zapisa pojedine stvarne klase, dok vrijednosti izvan glavne dijagonale prikazuju pogrešne klasifikacije.

Vrijednosti mjera uspješnosti za pojedine tipove anomalija nalaze u tablici 5.1 s makro prosjekom tih mjera.

Tablica 5.1 Mjere uspješnosti višeklasnog modela na umjetno generiranom ispitnom skupu

Tip anomalije	Broj primjera	Preciznost	Odziv	F1-mjera
Standard	7165	0,98693	0,99135	0,98914
Nestabilni ispiti	271	0,9456	0,83395	0,88627
Konfiguracijski Pomak	220	1,0	0,96818	0,98383
Sigurnosna Anomalija	152	0,65822	0,68421	0,67097
Tihi neuspjeh	38	0,82051	0,842105	0,83117

Makro prosjek	7846	0,88225	0,86395	0,87227
---------------	------	---------	---------	---------

Ukupna točnost modela na umjetno generiranom ispitnom skupu iznosi 0,99184. Točnost nad anomalijским zapisima iznosi 0,92511, što znači da je model u tom udjelu anomalijских zapisa ispravno odredio pripadnu anomalijску klasu. Makro prosjek preciznosti iznosi 0,88225, makro prosjek odziva 0,86395, a makro prosjek F1-mjere 0,87227. Budući da uravnotežena točnost u ovom višeklasnom slučaju odgovara makro prosjeku odziva po klasama, uravnotežena točnost iznosi 0,86395.

Rezultati pokazuju da model vrlo uspješno prepoznaje standardne zapise i konfiguracijski pomak. Za standardne zapise postignute su visoke vrijednosti preciznosti, odziva i F1-mjere, što pokazuje da model rijetko pogrešno označava standardne zapise i da dobro prepoznaje većinsku klasu. Konfiguracijski pomak također je vrlo dobro prepoznat, s preciznošću 1,0 i odzivom 0,96818.

Slabiji rezultati vidljivi su kod sigurnosnih anomalija, gdje F1-mjera iznosi 0,67097. To upućuje na to da sigurnosne anomalije u umjetno generiranom skupu imaju obrasce koje model slabije razlikuje od ostalih klasa. Nestabilni ispiti imaju visoku preciznost, ali niži odziv, što znači da su predikcije modela za tu klasu relativno pouzdane, ali dio stvarnih nestabilnih ispita ostaje neprepoznat. Kod tih neuspjeha preciznost i odziv imaju slične vrijednosti, što upućuje na uravnoteženije ponašanje modela za tu klasu. Ovi rezultati potvrđuju da se višeklasni model može uspješno naučiti na kontrolirano generiranim zapisima, ali istodobno pokazuju da ukupna točnost nije dovoljna za procjenu kvalitete modela.

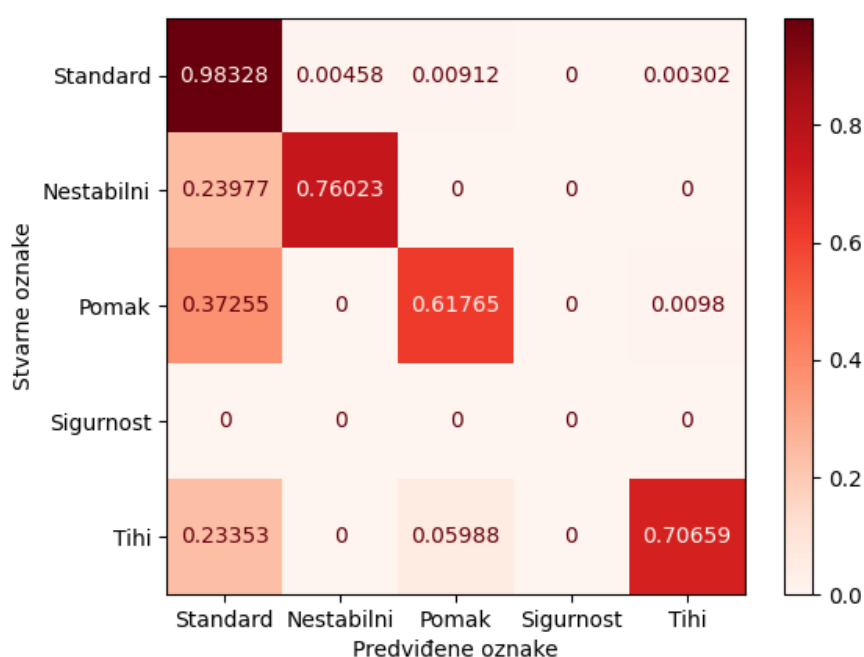
Zbog velikog broja standardnih zapisa, visoka ukupna točnost može prikriti slabiju detekciju pojedinih anomalijских klasa. Stoga se rezultat tumači zajedno s makro prosjekom, uravnoteženom točnošću, F1-mjerom po klasama i matricom zabune. Uravnotežena točnost od 0,86395 pokazuje da model ne ostvaruje visoku uspješnost samo zbog većinske standardne klase, ali niža F1-mjera sigurnosnih anomalija pokazuje da uspješnost nije jednaka za sve tipove anomalija.

5.2.2. Rezultati na skupu podataka GHALogs

Višeklasni model transformera nije vezan uz radni tok na kojem je učen i može se primijeniti na bilo koji radni tok. Nakon ispitivanja na umjetno generiranom skupu, višeklasni model evaluiran je na označenom podskupu stvarnih zapisa iz skupa

GHALogs. Time se provjerava može li se model primijeniti na zapise koji potječu iz stvarnih GitHub Actions radnih tokova. Ti se zapisi razlikuju od umjetno generiranih zapisa po strukturi, rječniku, korištenim alatima i obrascima izvođenja. Evaluacija na označenom podskupu GHALogs stoga daje ograničenu procjenu prenosivosti modela iz kontroliranog okruženja na odabrane stvarne GitHub Actions zapise.

Matrica zabune za dodatno trenirani model ispitana na označenom podskupu podataka iz GHALogs prikazana je na slici 5.3. Matrica je normalizirana po redcima, pa vrijednosti u pojedinom retku prikazuju kako su zapisi jedne stvarne klase raspodijeljeni po klasama koje je predvidio model.



Slika 5.3 Matrica zabune na ispitnome skupu GHALogs

Preciznost, odziv i F1-mjera su prikazane u tablici 5.2. Ukupna točnost na ispitnome skupu iznosi 0,97522, a točnost nad anomalijским zapisima iznosi 0,716639.

Tablica 5.2 Mjere uspješnosti višeklasnog modela na ispitnome skupu podataka GHALogs

Tip anomalije	Broj primjera	Preciznost	Odziv	F1-mjera
Standard	19856	0,99202	0,98343	0,98771
Nestabilni ispit	171	0,60185	0,76023	0,67183
Konfiguracijski Pomak	102	0,23864	0,61765	0,34426

Tihi neuspjeh	334	0,7893	0,70658	0,74565
Makro prosjek	20463	0,65545	0,767	0,68736

Makro prosjek preciznosti iznosi 0,65545, makro prosjek odziva 0,767, a makro prosjek F1-mjere 0,68736. Makro prosjeci za podskup GHALogs izračunani su nad klasama za koje postoje označeni primjeri u tom podskupu. Budući da u podskupu GHALogs nisu pronađene sigurnosne anomalije, ta klasa nije uključena u izračun mjera za GHALogs evaluaciju. Zbog toga makro prosjeke podskupa GHALogs treba uspoređivati s makro prosjecima umjetno generiranog skupa uz napomenu da nisu izračunani nad potpuno istim skupom klasa.

Uravnotežena točnost na podskupu GHALogs iznosi 0,767 i računa se kao makro prosjek odziva po klasama uključenima u evaluaciju tog podskupa. Budući da u označenom podskupu GHALogs nisu pronađene sigurnosne anomalije, ta klasa nije uključena u izračun mjera za GHALogs evaluaciju.

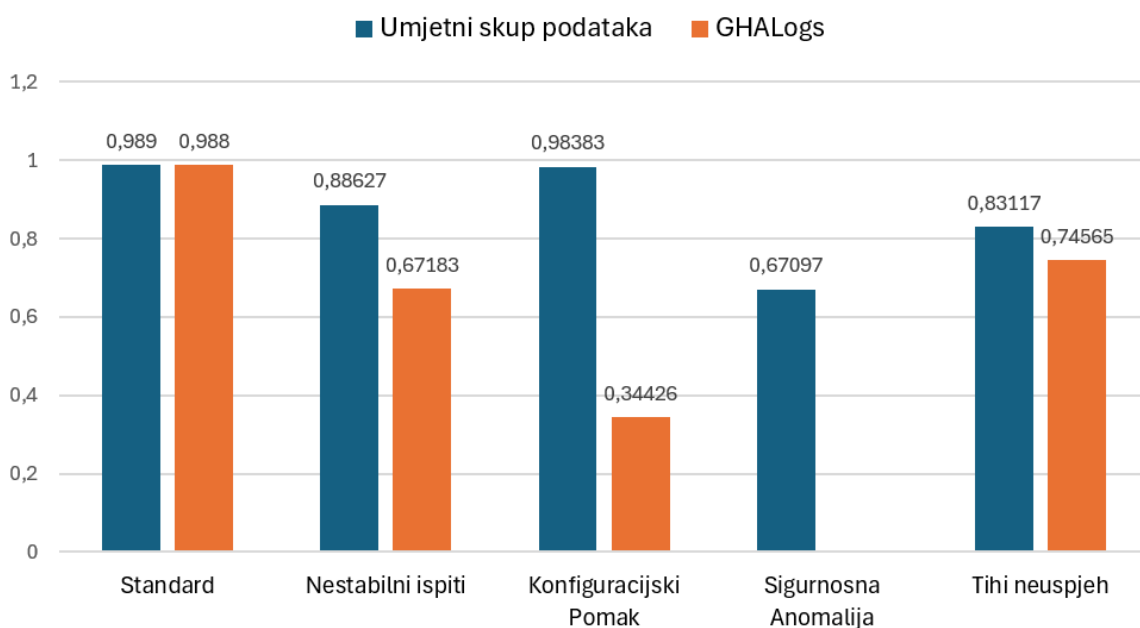
Uravnotežena točnost pokazuje da model zadržava dio sposobnosti prepoznavanja različitih klasa i na stvarnim zapisima. Međutim, ta se vrijednost ne smije tumačiti samostalno jer uravnotežena točnost odražava prosjek odziva, ali ne prikazuje pouzdanost pozitivnih predikcija. To je posebno važno za konfiguracijski pomak, kod kojeg je odziv 0,61765, ali je preciznost samo 0,23864. Zbog toga se uspješnost na podskupu GHALogs mora tumačiti zajedno s F1-mjerom, preciznošću po klasama i matricom zabune.

Rezultati na skupu GHALogs pokazuju očekivani pad uspješnosti u odnosu na umjetno generirani skup podataka. Iako ukupna točnost na ispitnom skupu GHALogs iznosi 0,97522, uravnotežena točnost od 0,767 i makro F1-mjera od 0,68736 pokazuju da visoka ukupna točnost prikriva znatno zahtjevniju detekciju manjinskih anomalijских klasa u stvarnim zapisima.

Model najuspješnije prepoznaje standardne zapise, za koje F1-mjera iznosi 0,98771. Time se pokazuje da model dobro zadržava sposobnost razlikovanja standardnih zapisa od anomalijских i nakon prijelaza na stvarne zapise. Za tihe neuspjehe postignuta je F1-mjera 0,74565, što pokazuje relativno stabilno ponašanje modela za tu klasu. Nestabilni ispiti imaju F1-mjeru 0,67183, pri čemu je odziv viši od preciznosti. To znači da model pronalazi velik dio stvarnih nestabilnih ispita, ali pritom dio drugih zapisa pogrešno označava kao nestabilne ispite.

Najslabiji rezultat postignut je za konfiguracijski pomak, gdje preciznost iznosi 0,23864, odziv 0,61765, a F1-mjera 0,34426. Takav odnos preciznosti i odziva pokazuje da model relativno često označava zapise kao konfiguracijski pomak, ali značajan dio tih predikcija nije točan. Razlog može biti u tome što stvarni zapisi često sadrže tekstualne tragove promjene verzija, upozorenja, zastarjelih paketa ili konfiguracijskih poruka koje nisu nužno stvarni konfiguracijski pomak u smislu oznaka korištenih u ovom radu.

Slika 5.4 prikazuje usporedbu F1-mjere po klasama na umjetno generiranom ispitnom skupu i na označenom podskupu GHALogs. Usporedba pokazuje da se F1-mjera standardne klase gotovo ne mijenja, dok se kod anomalijских klasa pojavljuje izraženiji pad uspješnosti. Najveći pad vidljiv je kod konfiguracijskog pomaka, što potvrđuje da su stvarni obrasci konfiguracijskih promjena u GHALogs zapisima raznolikiji od kontrolirano umetnutih obrazaca u umjetno generiranom skupu. Sigurnosne anomalije nisu prikazane za podskup GHALogs jer u označenim stvarnim zapisima nisu pronađeni primjeri te klase.



Slika 5.4 Usporedba F1-mjere po klasama na umjetno generiranom skupu i GHALogs podskupu

Rezultati na skupu GHALogs dobiveni bez dodatnog učenja autoenkodera. Time se pokazuje da se reprezentacije zapisa naučene na umjetno generiranom skupu mogu u određenoj mjeri primijeniti i na stvarne zapise drukčijeg oblika. Ipak, pad uspješnosti višeklasnog modela potvrđuje da bi za robusniju primjenu na stvarnim

radnim tokovima bilo potrebno prikupiti i označiti veći broj raznolikih stvarnih primjera.

Sažetak glavnih rezultata višeklasnog modela prikazan je u tablici 5.3. Tablica objedinjeno prikazuje ukupnu točnost, točnost nad anomalijским zapisima, makro F1-mjeru i uravnoteženu točnost za umjetno generirani skup podataka i podskup GHALogs.

Tablica 5.3 Sažetak glavnih rezultata višeklasnog modela

Skup podataka	Ukupna točnost	Točnost nad anomalijским zapisima	Makro F1-mjera	Uravnotežena točnost
Umjetno generirani skup	0,99184	0,92511	0,87227	0,86395
Podskup GHALogs	0,97522	0,716639	0,68736	0,767

5.2.3. Analiza pogrešaka višeklasnog modela

Analiza pogrešaka višeklasnog modela pokazuje da se prijelaz s umjetno generiranih na stvarne zapise najviše očituje u slabijem razlikovanju pojedinih anomalijских klasa.

Ova razlika je najizraženija kod mjere preciznosti za klasu konfiguracijski pomak gdje preciznost na stvarnim podacima iznosi 0,23864, a na umjetno generiranim 1,0. Značajan je i pad u odzivu iste klase koji na stvarnim podacima iznosi 0,61765, u usporedbi s 0,96818 na umjetno generiranom skupu podataka. Navedeni rezultat ukazuje da se obrasci konfiguracijskog pomaka u stvarnim uvjetima značajno razlikuju od jednostavne promjene verzije paketa implementirane u umjetnom skupu podataka.

Prilikom prijelaza na stvarne podatke model je pokazao najveću uspješnost u detekciji tih neuspjeha. F1-mjera za ovu klasu na stvarnim podacima iznosi 0,74565, što je i dalje manje od vrijednosti ostvarene na umjetno generiranom skupu (0,83117). Unatoč blagom padu performansi, ovi rezultati potvrđuju da model

uspješno prepoznaje anomalije u obliku tihih neuspjeha i pri primjeni na označenom podskupu GHALogs.

Klasifikacija nestabilnih ispita je značajno manja na stvarnim podacima s preciznošću 0,60185 i odzivom 0,76023, a na umjetnim podacima preciznost iznosi 0,82051 i odziv 0,842105. Pogrešno klasificirani zapisi pretežito se kategoriziraju kao standardni zapisi. Iz čega se može zaključiti da model rijetko međusobno miješa različite klase anomalija, no posljedično pokazuje veću vjerojatnost propuštanja, odnosno ne otkrivanja stvarnih anomalija u stvarnom okruženju.

Detekcija sigurnosnih anomalija ostvarila je najnižu učinkovitost na umjetno generiranom skupu podataka, uz vrijednost F1-mjere od 0,67097. Evaluacija sposobnosti modela u detekciji sigurnosnih anomalija na stvarnom skupu podataka nije bila izvediva zbog nedostatka označenih primjera. S obzirom na općeniti pad vrijednosti mjera uspješnosti prilikom prijelaza na stvarne podatke kod ostalih klasa, opravdano je pretpostaviti da bi se slična degradacija performansi očitovala i u segmentu detekcije sigurnosnih anomalija.

5.3. Evaluacija rekonstrukcijskog modela

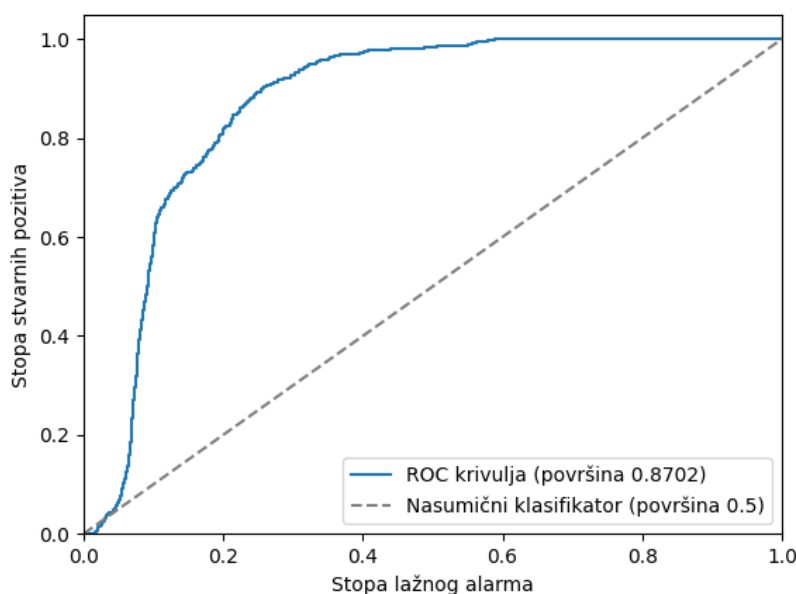
Kod rekonstrukcijskog modela detekcija anomalija ovisi o pragu koji se postavi na pogrešku kod rekonstrukcije. Rekonstrukcijski model transformera binarno određuje radili se o anomaliji ili ne. Zbog toga se za evaluaciju takvog modela koristi ROC krivulja. ROC krivulja i AUC vrednuju sposobnost modela razlikovanja standardnih i anomalijских primjera kroz cijeli raspon mogućih pragova. Time se ne odabire niti zasebno potvrđuje operativni prag $T_{op}=0,38852$ korišten u integriranoj GitHub Akciji.

Rekonstrukcijski model transformera ovisi o radnome toku na kojem je učen, ako se primjeni na novi nepoznati radni tok greška kod rekonstrukcije biti će velika za svaki zapis. Zbog toga se rekonstrukcijski model ispituje samo na umjetno generiranom skupu podataka. Označeni podskup GHALogs zapisa sastoji se samo od pokretanja radnih tokova koji sadrže anomalije zbog toga je rekonstrukcijski model ispitan samo na umjetno generiranom skupa podataka. Moguće je učiti model na zapisima bez anomalija iz GHALogs, ali tada bi za ispitivanje bilo potrebno umjetno ubaciti anomalije što opet ne daje sliku o evaluaciji u stvarnim uvjetima. Zbog toga se rezultat rekonstrukcijskog modela tumači kao provjera mogućnosti detekcije

odstupanja u kontroliranom radnom toku, a ne kao dokaz njegove prenosivosti na neovisne stvarne GitHub Actions radne tokove.

Za eksperimentalni prikaz sposobnosti razdvajanja standardnih i anomalijских zapisa korištena je ROC krivulja i površina ispod nje. Budući da je za operativnu primjenu u integracijskom toku potrebna jedna odluka, dodatno je određen operativni prag T_{op} . Prag je izračunat kao prosječna vrijednost rekonstrukcijskih pogrešaka standardnih zapisa iz skupa za učenje na koju se dodaje vrijednost standardne devijacije pomnožen s dva. Ovaj prag obuhvaća 97.72 % rekonstrukcijskih pogrešaka standardnih zapisa, a dobivena vrijednost iznosi $T_{op} = 0,38852$. Zapis se u integriranoj primjeni označava anomalijским kada njegova rekonstrukcijska pogreška zadovoljava $e > T_{op}$. Ta je vrijednost pohranjena je u konfiguraciji GitHub akcije i primjenjuje se u skripti detect.py.

Prikaz krivulje ROC za rekonstrukcijski model transformera se nalazi na slici 5.5. Na grafu ROC krivulje prikazan je nasumični klasifikator čija površina pod krivljom iznosi 0,5. Na slici je plavom bojom označena ROC krivulja transformera ispod koje površina iznosi 0,8702. Isprekidana siva crta predstavlja ROC krivulju nasumičnog klasifikatora.



Slika 5.5 ROC krivulja rekonstrukcijskog modela

ROC krivulja je dobivena kretanjem vrijednosti praga između 3,70604 i 0,0008. Ove vrijednosti predstavljaju najveću i najmanju rekonstrukcijsku pogrešku na ispitnom skupu. Prag prvo propitam najveću vrijednost rekonstrukcijske pogreške, za taj prag

se računaju stopa lažnog alarma i odziv. Zatim se prag pomiče na sljedeću najveću rekonstrukcijsku pogrešku za koju se računa odziv i stopa lažnog alarma. Prag se tako pomiče sve dok ne dođe do najmanje vrijednosti rekonstrukcijske pogreške.

5.4. Rasprava rezultata i ograničenja

Usporedbom mjera uspješnosti višeklasnog modela na umjetno generiranom skupu podataka i skupu GHALogs vidljiv je očekivani pad uspješnosti modela na stvarnim podacima. Kod oba skupa podataka model postiže najvišu točnost pri klasifikaciji standardnih zapisa, što ukazuje na nisku stopu lažno pozitivnih rezultata, odnosno model rijetko klasificira normalno ponašanje kao anomaliju.

Pogrešno klasificirani zapisi većinom se odnose na anomalije koje su prepoznate kao standardni zapisi, što upućuje da model rijetko međusobno miješa različite klase anomalija. U kontekstu detekcije anomalija važno je razlikovati dvije vrste pogrešaka: lažno pozitivne nalaze, koji mogu nepotrebno opteretiti razvojni tim, i lažno negativne nalaze, kod kojih stvarna anomalija ostaje neprepoznata. Zbog toga se rezultati ne mogu tumačiti samo jednom mjerom, nego se zajedno razmatraju preciznost, odziv, F1-mjera i matrica zabune. Kod skupa GHALogs uočava se značajna razlika između odziva (0,61765) i preciznosti (0,23864) za klasu konfiguracijski pomak, dok je kod ostalih klasa anomalija ta razlika znatno manja.

Makro prosjek preciznosti je za 0,2268 veći na umjetno generiranom skupu podataka od skupa podataka GHALogs. To ukazuje na to da model na stvarnim podacima pokazuje veću sklonost generiranju lažno pozitivnih rezultata, dok je na umjetno generiranim podacima izraženija vjerojatnost pojave lažno negativnih rezultata. Makro prosjek F1-mjere na umjetnom skupu iznosi 0,87227, dok na stvarnim podacima iznosi 0,68736. Pad vrijednosti pokazuje da je prijelaz s kontrolirano generiranih na stvarne zapise zahtjevniji, osobito zbog veće heterogenosti zapisa i ograničenog broja označenih primjera. Stoga se rezultati na podskupu GHALogs tumače kao provjera primjenjivosti modela na stvarnim zapisima u okviru provedenog eksperimentalnog protokola. Razliku u ponašanju modela na dvama skupovima podataka potvrđuje i uravnotežena točnost. Na umjetno generiranom skupu ona iznosi 0,86395, dok na podskupu GHALogs iznosi 0,767. Budući da u višeklasnom slučaju odgovara makro prosjeku odziva po

klasama, uravnotežena točnost pokazuje koliko model u prosjeku uspješno prepoznaje pojedine klase, neovisno o njihovoj zastupljenosti. Međutim, budući da ne uzima u obzir preciznost pozitivnih predikcija, ovu je mjeru potrebno tumačiti zajedno s preciznošću, F1-mjerom i matricom zabune kako bi se dobila cjelovita slika uspješnosti modela.

Važno je naglasiti da su rezultati na GHALogs dobiveni bez dodatnog učenja autoenkodera što pokazuje da je on otporan na promijene oblika zapisa.

Ograničenje višekratnog modela proizlazi iz količine podataka na kojima je učen i raznolikosti obuhvaćenih anomalija. Da bi model mogao dobro razlikovati različite oblike anomalija skup podataka na kojem je model učen trebao bi obuhvaćati rijetke anomalijama iz različitih produkcijskih okruženja i programskih jezika. U stvarnim radnim tokovima anomalije se pojavljuju rijetko, što otežava prikupljanje takvih podataka.

Analizom ROC krivulje rekonstrukcijskog modela uočava se da je u početnom segmentu njezina vrijednost niža od krivulje nasumičnog klasifikatora, dok je za sve ostale klasifikacijske pragove viša. Površina ispod krivulje je za 0,0937 manja površine ROC krivulje za model iz [6] koja iznosi 0,9639. Ključno ograničenje rekonstrukcijskog modela jest njegova ovisnost o učenju na podacima koji ne sadrže anomalije. Navedeno ograničenje uvjetovalo je evaluaciju modela isključivo na umjetno generiranom skupu podataka, s obzirom na to da stvarni podaci ne nude uparene primjere identičnih radnih tokova s anomalijama i bez njih.

Provedena evaluacija pokazuje da je GHA-AIOps pristup primjenjiv kao osnova za automatiziranu analizu GitHub Actions zapisa. Višeklasni model prikladan je kada je korisniku potrebno dati oznaku tipa anomalije, dok je rekonstrukcijski model prikladan za otkrivanje odstupanja od naučenog standardnog ponašanja. Kombinacija tih dvaju modela omogućuje klasifikacijski i rekonstrukcijski pogled na anomalije, čime se povećava fleksibilnost pristupa u odnosu na uporabu samo jednog modela. Za daljnje poboljšanje potrebno je proširiti označeni skup stvarnih zapisa, uključiti više repozitorija i radnih tokova, sustavno pratiti stopu lažnih uzbuna po radnom toku te evaluirati modele na strože odvojenim skupovima podataka.

6. Zaključak

U ovom radu razrađen je i evaluiran pristup za detekciju anomalija u zapisima GitHub Actions radnih tokova. Polazište rada bio je problem kasnog uočavanja kvarova, degradacija i rizičnih obrazaca u CI/CD cjevovodima, osobito u slučajevima kada završni status radnog toka ne daje potpunu sliku o pouzdanosti i sigurnosti izvedenog procesa.

Anomalije koje se pojavljuju u zapisima radnih tokova se mogu dijeliti na konfiguracijski pomak, nestabilni ispiti, sigurnosne anomalije i tihe neuspjehe. Za primjenu dubokih modela potrebno je dodijeliti numeričke reprezentacije zapisima izlučivanjem značajki. Postoje različiti pristupi izlučivanju značajki iz nestrukturiranih zapisa. Parsiranjem je moguće svesti nestrukturirane podatke na konačan broj predložaka. Što su zapisi više različiti to broj predložaka raste. Nedostatak parsiranja je različito reprezentiranje semantički sličnih uzoraka i teška prilagodba promjeni oblika zapisa. Dodjela numeričkih reprezentacija zapisima moguća je korištenjem dubokih modela kao što je Autoenkoder. Autoenkoder je korišten kako bi se smanjila ovisnost o ručno definiranim predlošcima zapisa i omogućila izgradnja numeričkih reprezentacija iz tekstualne strukture zapisa. Rezultati na označenom podskupu GHALogs upućuju na to da se reprezentacije zapisa naučene na umjetno generiranom skupu mogu u određenoj mjeri primijeniti i na stvarne zapise drukčijeg oblika. Međutim, ti rezultati ne dokazuju opću robusnost na sve oblike GitHub Actions zapisa, nego pokazuju ograničenu prenosivost u okviru ispitivanog podskupa.

Rekonstrukcijski model transformera je ovisan o radnome toku u kojem se koristi. Potrebno ga je periodički ponovno trenirati kako bi se prilagodio promjenama u radnom toku. Ispitivanje višeklasnog modela tranformera na skupu podataka GHALogs je pokazalo da je uspješnost na stvarnim podacima manja nego na umjetnom skupu podataka, ali se model i dalje može primijeniti na različite radne tokove. GitHub Akcije omogućavaju integriranje ovog modela u radni tok tako što pružaju mogućnost izrade javno dostupne Akcije koja se može uključiti u bilo koji radni tok.

Predloženi pristup evaluiran je na umjetno generiranom označenom skupu podataka i na označenom podskupu stvarnih GHALogs zapisa. Na umjetno generiranom skupu višeklasni model postigao je ukupnu točnost od 0,99184, točnost nad anomalijским zapisima od 0,92511 i makro F1-mjeru od 0,87227. Na označenom podskupu GHALogs zapisa ukupna točnost iznosila je 0,97522, točnost nad anomalijским zapisima 0,716639, a makro F1-mjera 0,68736. Rekonstrukcijski model vrednovan je pomoću ROC krivulje, pri čemu je površina ispod krivulje iznosila 0,8702. Evaluacija na podskupu GHALogs pokazuje da se predloženi pristup može primijeniti i izvan kontrolirano generiranog okruženja, uz smanjenje uspješnosti koje upućuje na potrebu za većim i raznolikijim označenim skupom podataka.

Budući da su skupovi podataka izrazito neuravnoteženi, ukupna točnost ne daje potpunu sliku uspješnosti modela. Stoga se rezultati tumače prvenstveno kroz makro F1-mjeru, uravnoteženu točnost i rezultate po klasama, koji jasno pokazuju pad uspješnosti pri prijelazu s umjetno generiranih na stvarne GHALogs zapise.

U budućem radu ovaj pristup moguće je proširiti dodatnim tipovima anomalija koje višeklasni model prepoznaje. Za to je potrebno prikupiti i dodatno označiti veliki broj zapisa stvarnih radnih tokova. Dodatno je korisno razmotriti optimizaciju programskog rješenja kako bi se smanjilo vrijeme izvođenja i ovisnost o velikim vanjskim paketima.

Literatura

- [1] Workflow syntax for GitHub Actions. Poveznica: <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>;
- [2] Zheng, L., Li, S., Huang, X., Huang, J., Lin, B., Chen, J., Xuan, J., *Why Do GitHub Actions Workflows Fail? An Empirical Study*. ACM Transactions on Software Engineering and Methodology, 2025.
- [3] Saito, T., Rehmsmeier, M., *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. PLoS ONE 10(3): e0118432. (2015), <https://doi.org/10.1371/journal.pone.0118432>
- [4] Fawcett, T., *An Introduction to ROC Analysis*. Pattern Recognition Letters, 27 (8), (2006) str. 861–874. doi:10.1016/j.patrec.2005.10.010.
- [5] He, S., He, P., Chen, Z., Yang, T., Su, Y., Lyu, M. R., *A Survey on Automated Log Analysis for Reliability Engineering* ACM Computing Surveys, (2021), <https://doi.org/10.1145/3460345>
- [6] Zhang, X., Chai, X. Yu, M., Qiu, D. *Anomaly Detection Model for Log Based on LSTM Network and Variational Autoencoder*. 2023 4th International Conference on Information Science, Parallel and Distributed Systems (ISPDS), Guangzhou, China, (2023), str. 239-244
- [7] Hemalatha, C., Vatsavayi, V. K., *Detection of Anomalies in Log Data Using Transformer Variational Autoencoders*. 2025 IEEE 17th International Conference on Computational Intelligence and Communication Networks (CICN), Goa, India, (2025), str. 2180-2184
- [8] Abeyasinghe, A. M. A. D., Wijayanayake, W. M. J. I., *Systematic Literature Review of Explainable LLM-Powered ChatOps for CI/CD Pipeline Diagnostics and Developer Support*. 2025 International Conference on Advances in Technology and Computing (ICATC), Kelaniya, Sri Lanka, (2025), str. 1-6
- [9] Xu, W., Luo, J., Huang, T., Sui, K., Geng, J., Ma, Q., *LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation*. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Seoul, Korea, Republic of, (2025), str. 3742-3753.
- [10] Sun, S., Friberg, D., & Staron, M., *“Good” and “Bad” failures in industrial CI/CD—Balancing cost and quality assurance*. Lecture notes in computer science, (2025), str. 75-84

-
- [11] Luo, Q., Hariri, F., Eloussi, L., & Marinov, D., *An empirical analysis of flaky tests*. The 22nd ACM SIGSOFT International Symposium, (2014), str. 643-653
 - [12] Shrestha, R., Nur Ali, A. A., *Configuration Management in Kubernetes Environments: A GitOps Approach*. 2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC), Sharjah, United Arab Emirates, (2024), str. 497-502
 - [13] Rausch, T., Hummer, W., Leitner, P., Schulte, S., *An Empirical Analysis of Build Failures in the Continuous Integration Workflows of Java-Based Open-Source Software*. 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), Buenos Aires, Argentina, 2017, str. 345-355
 - [14] Sakeh, S. M., Al Ifat, M. N., Madhavji, N. H., Steinbacher, J., *A Threat-Oriented Study of API Security Challenges in CI/CD Pipelines*. 2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), Shenzhen, China, (2025), str. 1-8
 - [15] Jeyarajan, B., Murugan, A., Pandey, G., Pugazhenth, V. J., *AI for Predictive Monitoring and Anomaly Detection in DevOps Environments*. SoutheastCon 2025, Concord, NC, USA, (2025), str. 450-455
 - [16] About GitHub Agentic Workflows. Poveznica: <https://docs.github.com/en/copilot/concepts/agents/about-github-agentic-workflows>; Pristupljeno: 14. svibnja 2026.
 - [17] He, P., Zhu, J., Zheng, Z., Lyu, M.R., *Drain: An Online Log Parsing Approach with Fixed Depth Tree*. 2017 IEEE International Conference on Web Services (ICWS), Honolulu, HI, USA, (2017), str. 33-40
 - [18] Moriconi, F., Durieux, T., Falleri, J. R., Troncy, R., Francillon, A., *GHALogs: Large-Scale Dataset of GitHub Actions Runs*, 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Ottawa, ON, Canada, (2025), str. 669-673
 - [19] Mohammad, B., Wagner, S., *Anomaly Detection in Univariate Time-Series: A Survey on the State-of-The-Art*. ArXiv (Cornell University), (2020), <https://doi.org/10.48550/arxiv.2004.00433>.
 - [20] He, S., Zuh, J., He, P., Lyu, M. R., *Experience Report: System Log Analysis for Anomaly Detection*. 2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE), Ottawa, ON, Canada, (2016), str. 207-218
 - [21] Murphy, A., Larsson, D., Söderlund, F., Angelsmark, O., Eker, J., *Automated Log Message Embeddings*. 2024 11th International Conference on Internet of Things: Systems, Management and Security (IOTSMS), Malmö, Sweden, (2024), str. 192-197

-
- [22] Bank, D., Koenigstein, N., Giryas, R., *Autoencoders*. ArXiv.org, (2021), <https://doi.org/10.48550/arXiv.2003.05991>
 - [23] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., *Attention Is All You Need*. ArXiv.org, (2017), <https://doi.org/10.48550/arXiv.1706.03762>
 - [24] Guo, H., Yuan, S., Wu, X., *LogBERT: Log Anomaly Detection via BERT*. 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, (2021), str. 1-8

Dodatak A: Repozitorij, izvorni kod i reprodukcija rezultata

Ovaj dodatak opisuje repozitorij rada, predanu inačicu izvornog koda, podatkovne datoteke, naučene modele, skripte za učenje i ispitivanje te druge elektroničke artefakte povezane s radom. Repozitorij rada dostupan je na poveznici: <https://github.com/DominikOzvald/msc-dominik-ozvald-2026.git>

A.1 struktura

U ovom poglavlju je prikazana i opisana struktura repozitorija s izvornim kodom korištenim u radu. Programsko rješenje podijeljeno je u nekoliko glavnih direktorija. Cjelokupni izvorni kod nalazi se unutar direktorija src, dok su podaci, trenirani modeli i rezultati organizirani u zasebne cjeline radi modularnosti i lakše reproduktivnosti rezultata.

Direktorij src predstavlja jezgru implementacije i podijeljen je na tri primarna poddirektorija: GHA_AI Ops (GHA-AI Ops pristup), data_generation (generiranje umjetnih podataka) i integration (Integracija GHA-AI Ops pristupa u radni tok)

Src/GHA_AI Ops sadrži logiku za definiranje arhitektura dubokih modela, njihovo učenje i ispitivanje. U njemu se nalaze sljedeće datoteke i direktoriji.

- GHA_AI Ops/models/ - Arhitektura modela
 - convlstm.py: Sadrži implementaciju autoenkodera
 - transformer.py: Sadrži definicije razrede (engl. class) za višeklasni i rekonstrukcijski model transformera
 - embedder.py: Sadrži klasu omotača (engl. wrapper) oko autoenkodera koja služi za pravilno oblikovanje i pripremu tenzora prije prosljeđivanja koderu autoenkodera.
- GHA_AI Ops/utils/ - Pomoćne funkcije i procesiranje zapisa:
 - data.py: Sadrži funkcije za pretprocesiranje tekstualnih zapisa.
 - datasets.py: Implementira prilagođene razrede za učitavanje podataka
 - embeddings.py: Sadrži klasu zaduženu za dodjeljivanje numeričkih vrijednosti ASCII znakovima unutar zapisa

- train.py: Sadrži petlje za učenje modela
- GHA_AIOps /training/ - skripte za pokretanje učenja
 - convlstm.py: Pokreće proces učenja autoenkodera.
 - transformer.py: Koristi se za treniranje rekonstrukcijskog modela transformera.
 - tag_transformer.py: Pokreće učenje višeklasnog modela transformera
 - tuning.py: Sadrži logiku za dodatno učenje višeklasnog modela transformera na stvarnim podacima
- GHA_AIOps/evaluating/ Evaluacijske skripte:
 - transformer.py: Služi za ispitivanje rekonstrukcijskog modela.
 - tag_transformer.py: Služi za ispitivanje višeklasnog modela.

Src/data_generation/ sadrži skripte korištene za generiranje umjetnog skupa podataka. U njemu se nalaze sljedeće datoteke

- build.py: Simulira fazu izgradnje aplikacije
- test.py: Simulira izvođenje jediničnih testova
- package.json: Definira pakete koji se pojavljuju u konfiguracijskom pomaku.
- test_values.json: Sadrži ispitne vrijednosti koje se koriste za simulaciju nestabilnih ispita

Src/integration/ sadrži skripte za dohvat zapisa i integriranje njihove obrade u radni tok. U njemu se nalaze sljedeće datoteke

- action.yml: Definira GitHuba akciju i njezine korake
- fetch_logs.py: Dohvaća zapise i sprema ih u virtualno okruženje
- detect.py: Učitava model i koristi ih za detekciju anomalija i izradu izvještaja

Direktorij data organiziran je u dvije cjeline s obzirom na podrijetlo i namjenu podataka. Data/dummy/ sadrži umjetno generirane podatke podijeljene na skupove za učenje, ispitivanje i učenje rekonstrukcijskog modela. Skup namijenjen rekonstrukcijskom modelu ne sadrži anomalije. Data/gha/ sadrži stvarne, označene zapise iz skupa GHALogs podataka, podijeljene na dio za učenje i ispitivanje.

Unutar direktorija models nalaze se sačuvane težine obučanih modela u .pt formatu, nazvane prema arhitekturi. U ovom direktoriju nalaze se sljedeći artefakti:

- ConvLSTM_E_32_H_196_L_128.pt: Sačuvani model autoenkodera.

- RecTransformer_DE_2_H_2_F_1024.pt: Rekonstrukcijski model transformera
- TaggedTransformer_E_2_H_2_F_1024_D_128.pt: Višeklasni model prije dodatnog učenja.
- TaggedTransformer_E_2_H_2_F_1024_D_128_tuned.pt: Višeklasni model nakon dodatnog učenja (finog podešavanja) na skupu GHALogs podataka

Direktorij results sadrži rezultate evaluacije modela. Datoteke u svojem nazivu sadrže ime pripadajućeg modela radi lakše identifikacije. U direktoriju se nalaze slike koje prikazuju matrice zabune za višeklasne modele, kao i ROC krivulje za rekonstrukcijski model. Osim slika u direktoriju se nalaze .txt datoteke s numeričkim vrijednostima preciznosti, odziva, F1-mjere i njihovim makro prosjekom.

A.2 Upute za pokretanje i programska okolina

Repozitorij sadrži datoteku README.md s osnovnim informacijama o radu, preduvjetima i pokretanju ispitivanja. Za pokretanje koda potreban je Python 3.10 ili novija inačica te programski paketi definirani u datoteci requirements.txt. Ispitivanje naučenih modela pokreće se naredbama:

- `pip install torch numpy scikit-learn matplotlib`: instalacija Python paketa za pokretanje ispitivanja.
- `git clone https://github.com/DominikOzvald/msc-dominik-ozvald-2026.git` : kloniranje repozitorija s izvornim kodom i podcima
- `cd msc-dominik-ozvald-2026`: premještanje u repozitorij
- `python -m src.GHA_AIOps.evaluating.tag_transformer`: pokretanje ispitivanja višeklasnog modela
- `python -m src.GHA_AIOps.evaluating.transformer`: pokretanje ispitivanja rekonstrukcijskog modela

Pokretanjem navedenih naredba instalirat će se potrebni python paket za ispitivanje. Rezultat ispitivanja višeklasnog će biti matrica zabune kao ona u poglavlju 5 zajedno s ispisom preciznosti, odziva, F1-mjere i njihovog makro prosjeka na standardni izlaz. Kod ispitivanja rekonstrukcijskog modela rezultat će bit slika ROC krivulje kao ona u poglavlju 5.

Dodatak B: LSTM ćelija u koderu i dekoderu autoenkodera

Ovaj dodatak daje matematički opis LSTM ćelije koja se u radu koristi u koderu i dekoderu autoenkodera opisanog u potpoglavlju 4.2.1. Opis nije zaseban doprinos rada, nego služi kao tehnička podloga za razumijevanje načina na koji se tekstualni zapisi prevode u vektorske reprezentacije.

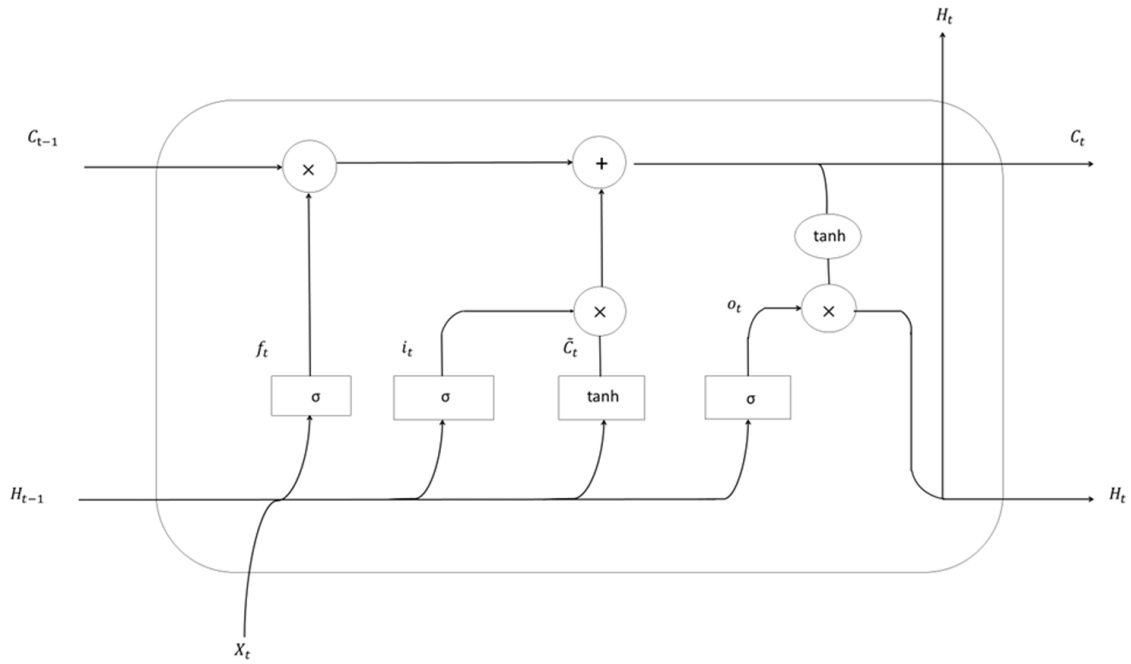
Poput ostalih povratnih neuronskih mreža (engl. Recurrent Neural Networks – RNN), LSTM mreže specijalizirane su za obradu podataka strukturiranih u obliku nizova. Budući da ulazni nizovi u LSTM ćeliju mogu biti varijabilne duljine, ova je arhitektura izrazito pogodna za analizu nestrukturiranih zapisa koji nastaju unutar CI/CD cjevovoda.

Povratne neuronske mreže sadrže unutarnje stanje koje sprema aktivacije prijašnjeg koraka u vremenskom nizu te se koristi za aktivaciju trenutnog koraka. LSTM osim memorijskih blokova za spremanje stanja koristi posebna vrata koja upravljaju tokom informacija. Vrata se dijele na ulazna vrata, izlaza vrata i vrata zaboravljanja označena na slici B.1 s i_t , o_t , i f_t . Za promjenu unutarnjeg stanja C_t potrebna je podrška ulaznih vrata i_t i vrata zaboravljanja f_t . Novo unutarnje stanje se mijenja prema izrazu (9).

$$C_t = f_t \otimes C_{t-1} + i_t \otimes \hat{C}_t \quad (9)$$

Gdje $\otimes$ predstavlja množenje matrice po elementima, a $\hat{C}$ predstavlja doprinos stanju ćelije. Vrata zaboravljanja služe kako bi pri svakome koraku odredile koje informacije iz prijašnjeg stanja ostaju u trenutnom staju. Izlaz vrata zaboravljanja se računa po izrazu (10).

$$f_t = \sigma(W_f[H_{t-1}, X_t] + b_f) \quad (10)$$



Slika B.1 Prikaz LSTM ćelije [6]

Gdje W_f i b_f predstavljaju težinsku matricu i pomak vrata zaboravljanja. H_{t-1} predstavlja izlaz LSTM ćelije za prijašnji podatak, a X_t predstavlja trenutni ulaz u LSTM ćeliju. Koristi se sigmoida (engl. sigmoid) kao aktivacijska funkcija označena sa σ . Ulazna vrata računaju svoj izlaz na isti način kao vrata zaboravljanja samo koriste različitu težinsku matricu i pomak označen s W_i i b_i u izrazu (11). Ulazna vrata određuju koje će informacije iz ulaznog podatka X_t ući u trenutno stanje ćelije C_t .

$$i_t = \sigma(W_i[H_{t-1}, X_t] + b_i) \quad (11)$$

Doprinos stanju ćelije se računa slično kao ulazna vrata i vrata zaboravljanja. Koristi vlastitu težinsku matricu i pomak W_c i b_c te umjesto sigmoide koristi tangens hiperbolni kao aktivacijsku funkciju prikazano u izrazu (12).

$$\hat{C}_t = \tanh(W_c[H_{t-1}, X_t] + b_c) \quad (12)$$

Izlazna vrata se koriste kako bi se odredio izlaz iz LSTM ćelije za trenutni podatak. Izlaz izlaznih vrata računa se isto kao kod prijašnjih vrata samo s vlastitim težinskim matricama i pomakom prikazanom u izrazu (13). Konačni izlaz iz LSTM ćelije se računa pomoću trenutnog stanja ćelije i izlaznih vrata prema izrazu (14).

$$o_t = \sigma(W_o[H_{t-1}, X_t] + b_o) \quad (13)$$

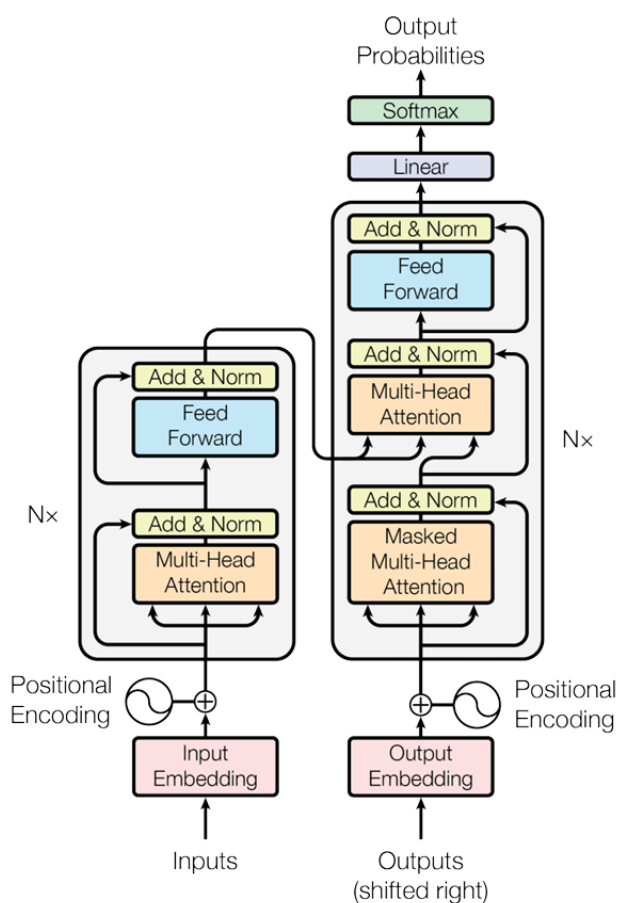
$$H_t = o_t \otimes \tanh(C_t) \quad (14)$$

U GHA-AIOps pristupu LSTM se ne koristi kao samostalan klasifikator anomalija, nego kao dio autoenkodera za dobivanje vektorske reprezentacije pojedinog retka zapisa. Na ulaz u LSTM ćelije se predaje niz vektorskih reprezentacija znakova zapisa koje se dobivaju iz matrice ugradnje. Izlaz LSTM grane spaja se s izlazom konvolucijske grane, čime se u jednoj reprezentaciji kombiniraju lokalni obrasci znakova i slijedni kontekst zapisa. Tako dobiveni vektori zatim se grupiraju u klizne prozore i predaju modelima transformera opisanima u poglavlju 4.3.

Dodatak C: Arhitektura transformera u modelima za detekciju anomalija

U ovom se radu primjenjuju modeli temeljeni na arhitekturi transformera. Transformeri se često koriste u području obrade prirodnoga jezika prvenstveno zbog svoje sposobnosti učinkovitog modeliranja dugoročnih ovisnosti i konteksta u kojem se pojedine riječi nalaze. Unutar predloženog GHA-AIOps pristupa, transformeri se koriste za analizu i hvatanje složenih odnosa između redaka zapisa unutar pomičnog prozora. Izvorna arhitektura prilagođena je za detekciju anomalije, što obuhvaća rekonstrukciju zapisa kod rekonstrukcijskog modela te klasifikaciju anomalija kod višeklasnog modela. U nastavku ovog dodatka opisuje se standardna arhitektura transformera.

Transformer je model temeljen na mehanizmu samopažnje koji se ističe kod obrade podataka poredanih u niz. Prikaz njegove arhitekture nalazi na slici C.1.



Slika C.1 Prikaz arhitekture transformera [23]

Transformer se također sastoji od kodera i dekodera. Koder se sastoji od N jednakih slojeva. Svaki sloj sadrži dva podsloja od čega je jedan višeglavi mehanizam samopažnje, a drugi potpuno povezana neuronska mreža. Korištena je rezidualna veza nakon svakog podsloja nakon čega slijedi normalizacija slojeva. Dekoder također sadrži N jednakih slojeva, koji za razliku od kodera sadrže još jedan podsloj s mehanizmom samopažnje koji prihvaća izlaze iz kodera. Mehanizam samopažnje promijenjen je kod dekodera na način da koristi maskiranje kako bi pretpostavka vrijednosti u trenutku t mogla ovisiti samo o njemu prethodnim ulazima u nizu.

Podaci koje transformer obrađuje ne sadrže u sebi informacije o njihovom vremenskom rasporedu. Na ulazne vektore transformera se dodaje pozicijsko kodiranje koje se računa po izrazima (15) i (16).

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (15)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (16)$$

Pos predstavlja poziciju ulaznog vektora u nizu, i predstavlja poziciju vrijednosti u vektoru, a d je dimenzija vektora. Mehanizam samopažnje je zaslužan za određivanje odnosa podataka u nizu. Za svaku glavu mehanizama potpuno povezani sloj stvara vektor upita (Q), ključa (K) i vrijednosti (V) iz kojih se računa matrica pažnje prema izrazu (17).

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (17)$$

Dimenzija vektora ključa označena je s d_k . Izlazi mehanizma samopažnje se spajaju u jedan vektor i ulaze u potpuno povezani sloj. Konačan izlaz dekodera ulazi u potpuno povezani sloj nakon čega se koristi softmax aktivacija za određivanje klasifikaciju ulaza.

U implementiranom GHA-AIOps pristupu transformer se koristi u dvjema različitim ulogama. U višeklasnom modelu koristi se koderski dio transformera, a dobivena reprezentacija ulaznog slijeda prosljeđuje se potpuno povezanom sloju i softmax funkciji radi razvrstavanja zapisa u standardnu klasu ili jednu od anomalijских klasa. U rekonstrukcijskom modelu koriste se koder i dekodeer transformera kako bi se rekonstruirao ulazni slijed vektorskih reprezentacija zapisa. Anomalija se tada ne

određuje izravnom klasifikacijom softmax funkcijom, nego na temelju povećane rekonstrukcijske pogreške između ulaznog i rekonstruiranog slijeda.